Republic of Tunisia

Ministry Of Higher Education And Scientific Research

University Of Tunis El Manar

Higher Institute Of Computer Science

Final Internship Report

Presented for the purpose of obtaining

the National Bachelor's Degree in COMPUTER SCIENCE

Specialization : Software Engineering and Information Systems (CS)

By

Ayoub Krimi

ParkFlow POS: Integrated Point Of Sale and Backoffice System for Parking Operations

Professional supervisor: **Mr. Zied Bradai** — CTO (Chief Technology Officer)

Academic supervisor: **Ms. Hela Limam** — Assistant Professor

Conducted within Asteroidea

Asteroidea
Illuminates the ocean of innovations

Academic Year: 2025 - 2026

Republic of Tunisia

Ministry Of Higher Education And Scientific Research

University Of Tunis El Manar

Higher Institute Of Computer Science

Final Internship Report

Presented for the purpose of obtaining

the National Bachelor's Degree in COMPUTER SCIENCE

Specialization : Software Engineering and Information Systems (CS)

By

Ayoub Krimi

ParkFlow POS: Integrated Point Of Sale and Backoffice System for Parking Operations

Professional Supervisor: **Mr. Zied Bradai** — CTO (Chief Technology Officer)

Academic Supervisor: **Ms. Hela Limam** — Assistant Professor

Conducted within Asteroidea

Academic Year: 2025 - 2026

I authorize the student to submit their internship report for the defense.

Professional Supervisor, **Mr. Zied Bradai**

Signature and stamp

I authorize the student to submit their internship report for the defense.

Academic Supervisor, **Ms. Hela Limam**

Signature

Dedications

I dedicate this work to myself, in recognition of the effort, discipline, and perseverance invested throughout these years of study.
I also dedicate this work to my family, especially my parents, for their continuous support and for providing me with the opportunity to pursue my education under the best possible conditions.

Ayoub Krimi

Acknowledgements

I would like to express my sincere appreciation to my academic supervisor, **Ms. Hela Limam**, for her guidance and support in structuring and improving this report. Her feedback and corrections have been valuable in enhancing the overall quality of this work.

I would also like to thank **Asteroidea** for providing me with the opportunity to carry out this internship in a professional and supportive environment. I am grateful to the entire team for their warm welcome, their availability, and for sharing their knowledge and experience. Their support helped me improve my technical skills and better understand real-world development practices.

I would like to extend my special thanks to my professional supervisor, **Mr. Zied Bradai**, for his continuous guidance and for helping me make informed technical and architectural decisions throughout the project.

Finally, I would like to thank my parents for their constant support and encouragement, as well as for providing me with the means to pursue and complete my studies.

Ayoub Krimi

Table of Contents

List of Figures

List of Tables

List of Abbreviations

- **API** = Application Programming Interface
- **CI/CD** = Continuous Integration/Continuous Deployment
- **CRUD** = Create, Read, Update, Delete
- **CSS** = Cascading Style Sheets
- **CTO** = Chief Technology Officer
- **DB** = Database
- **EV** = Electric Vehicle
- **HTML** = HyperText Markup Language
- **HTTP** = HyperText Transfer Protocol
- **IP** = Internet Protocol
- **JSON** = JavaScript Object Notation
- **JWT** = JSON Web Token
- **LPN** = License Plate Number
- **ORM** = Object-Relational Mapping
- **PHP** = PHP Hypertext Preprocessor
- **PMS** = Parking Management System
- **POS** = Point of Sale
- **REST** = Representational State Transfer
- **SQL** = Structured Query Language
- **SSE** = Server Sent Events
- **UI** = User Interface
- **URL** = Uniform Resource Locator

General Introduction

In recent years, parking management has become a critical aspect of modern urban infrastructure. With the increasing number of vehicles and the diversity of parking environments such as airports, hospitals, and shopping centers, the need for efficient and reliable parking solutions has significantly grown. Organizations are now required to adopt intelligent systems that ensure smooth operations, better user experience, and high adaptability to different environments.

This internship was carried out at Asteroidea, a company specialized in providing innovative parking solutions. Asteroidea is composed of a team of experienced professionals dedicated to addressing complex and non-traditional parking challenges. With projects deployed in more than ten countries, the company focuses on delivering high-quality systems tailored to the needs of parking operators and their customers.

The objective of this project, entitled *"ParkFlow POS: Integrated Point Of Sale and Backoffice System for Parking Operations"*, is to design and develop a complete solution that facilitates parking payment operations and administrative management. The system includes a Point of Sale (POS) interface used by cashiers to process payments and manage vehicle exits, as well as a backoffice platform dedicated to system administration and configuration.

Before the development of this solution, several limitations were identified in existing systems, particularly regarding configuration management. The lack of dynamic configuration made it difficult to manage essential components such as cameras, barriers, and POS terminals. Tasks like assigning ports or updating system parameters were complex, time-consuming, and prone to errors.

To address these issues, a complete system was developed, including both frontend and backend components. The work involved the implementation of the POS interface for cashiers, its corresponding backend API, a dedicated printer service for receipts and reports, as well as a backoffice application with its own backend. This integrated approach allows better control, improved flexibility, and easier system management.

The technologies used in this project include React.js with TypeScript for frontend development, along with modern tools such as TailwindCSS and Zod. On the backend side, the system is built using Golang with the Gin framework and Bun ORM, while PostgreSQL is used as the database system.

The architecture also relies on NATS for event-driven and real-time communication between system components, as well as Server-Sent Events (SSE) for real-time status updates related to connected services such as printers.

This report is organized into four main chapters. The first chapter presents the project context, including the host organization and the problem statement. The second chapter focuses on requirements analysis and specification. The third chapter describes the system design. Finally, the fourth chapter details the implementation and realization of the developed solution.

Chapter 1

Project Context

Plan

Introduction

This chapter introduces the general context of the internship project and presents the environment in which the work was carried out. It first describes the host organization, Asteroidea, and its activity in the field of parking solutions. The chapter then presents the existing system and its main limitations, particularly in terms of configuration management and scalability.

After identifying the problem addressed by the project, the proposed solution and the main objectives of the internship are presented. This chapter therefore provides the necessary background for understanding the motivations behind the development of the new POS and backoffice system.

1.1 Host Organization

The internship was carried out at Asteroidea, a company based in Mégrine, Tunis, Tunisia, specializing in advanced parking solutions. The company focuses on designing and delivering innovative systems that address the needs of modern parking infrastructures across various sectors.

Asteroidea primarily operates in the field of parking management, offering both software and integrated solutions tailored to complex environments. Its expertise covers the development of intelligent systems that improve operational efficiency and enhance user experience.

The company provides a wide range of products and services, including parking management systems, Point of Sale (POS) systems, barrier control systems, camera integrations, and backoffice platforms for administration and monitoring. In addition, Asteroidea is also involved in emerging solutions such as electric vehicle (EV) charging platforms, demonstrating its adaptability to evolving technological trends.

Asteroidea's solutions are deployed across multiple domains, including airports, hospitals, shopping centers, private parking facilities, and universities. This diversity of clients reflects the flexibility and scalability of the systems developed by the company.

During this internship, the development work was carried out individually by the intern, under the supervision and guidance of a professional supervisor. This setup allowed for both autonomy in implementation and continuous technical support throughout the project.

1.2 Existing Solution and Limitations

Before the development of the proposed system, Asteroidea relied on an existing Point of Sale (POS) solution built using a PHP backend and a React frontend. While it covered basic parking payment operations, it showed several limitations that became harder to ignore as the company started targeting larger deployments.

One of the main issues was the lack of dynamic configuration. Parameters such as camera ports, barrier addresses, and terminal identifiers were hardcoded, making each new deployment a manual and error-prone process. Adapting the system to a different site required going into the code directly, which slowed down delivery and increased maintenance costs.

From a performance standpoint, PHP's synchronous execution model also raised concerns. For a facility processing millions of transactions per month, the ability to handle concurrent operations efficiently is critical, and the existing stack was not designed with that scale in mind.

These limitations made it clear that a new solution was needed, one that is configurable, maintainable, and capable of running reliably under sustained operational load.

1.3 Problem Statement

The limitations of the existing system became particularly significant in the context of a major upcoming deployment. Asteroidea is preparing the solution for the Cairo International Airport, one of its key clients, whose parking facility handles approximately two million transactions per month. At this scale, the weaknesses of the current system are no longer minor inconveniences but become real operational obstacles.

The hardcoded configuration model makes it nearly impossible to deploy and maintain multiple terminals across a large site without significant manual effort. Each change requires direct intervention in the codebase, which increases the risk of errors and slows down the entire operation.

Beyond configuration, the performance profile of the existing PHP backend is not suited for high-throughput environments. Processing a high volume of concurrent vehicle exits, managing real-time communication with cameras, barriers, and printers simultaneously, and keeping data synchronized with a central parking management system all require a backend that can handle

parallelism natively.

The problem is therefore twofold: the system lacks the flexibility needed for efficient multi-site deployment, and it lacks the performance foundation required to operate reliably at the scale of a major international airport.

1.4 Proposed Solution and Project Objectives

To address these limitations, a new solution was developed consisting of a modern POS system coupled with a backoffice platform. The goal was to build something that Asteroidea could confidently deploy at large-scale environments such as the Cairo International Airport, while also remaining flexible enough to serve smaller sites with different configurations.

The proposed solution introduces a modular and configurable architecture where system parameters are managed dynamically through the backoffice, without requiring any code changes between deployments. On the performance side, the backend was rebuilt using Go, a language designed for high concurrency, replacing the previous PHP implementation to better handle the throughput demands of large parking facilities.

The main objectives of this project can be summarized as follows:

- Introduce dynamic configuration mechanisms manageable through the backoffice platform.
- Rebuild the backend with Go to support high concurrency and better performance under load.
- Enhance the cashier interface to improve usability during daily operations.
- Support multiple parking management systems to increase deployment flexibility.
- Implement an offline mode using NATS, allowing the POS to continue processing payments when the connection with the PMS is lost.
- Improve overall maintainability through a clean, layered system architecture.

Chapter 2

Requirements Analysis and Specification

Plan

Introduction

This chapter focuses on the analysis and specification of the proposed parking management solution. It presents the functional and non-functional requirements identified during the internship period, based on discussions with the company team and observations of real parking operations.

The chapter also covers the technology stack selected to implement the solution, the main use cases of both the POS and backoffice platforms, and the key interaction scenarios that guided the system design. These elements form the foundation for the design and implementation described in the following chapters.

2.1 Project Actors

The proposed system involves several actors interacting with different components of the application. These actors include human users as well as external systems and services that contribute to the overall functionality of the platform.

Cashier: The cashier is the primary user of the Point of Sale (POS) system. This actor is responsible for processing payments, managing transactions, and handling vehicle exits. The cashier interacts directly with the POS interface to perform daily parking operations.

Administrator: The administrator interacts mainly with the backoffice platform, where they manage and configure system entities such as POS terminals, car parks, users, and roles. In certain situations, administrators may also interact with the POS system, particularly during the initial configuration of a new terminal or when performing privileged operations, such as opening barriers when the cashier does not have the required permissions.

Parking Management System (PMS): The PMS is an external system that communicates with the developed solution through a message broker. It is responsible for synchronizing data and providing information such as vehicle presence tickets generated by entry cameras, it stores and manages transactions performed by POS terminals. Additionally, it synchronizes shifts created by POS terminals locally.

External Services: The system also interacts with several external services. These include a printer service used by the POS to generate receipts and reports via HTTP communication, as

well as other services such as barrier control systems and camera services. For example, exit camera systems can trigger payment processes by calling specific endpoints of the POS system. Additional services, such as tariff management, are also integrated to extend system capabilities.

2.2 Functional Requirements

2.2.1 POS Requirements

The Point of Sale (POS) system is designed to support the daily operations of cashiers by providing a set of functionalities that enable efficient payment processing and transaction management.

The main functional requirements of the POS system are as follows:

- **Authentication:** The cashier must be able to log into the POS system using valid credentials.
- **Shift Management:** The cashier can start a new shift by providing an initial amount or continue a previously opened shift if it was not closed.
- **Payment Processing:** The cashier can process payments for vehicles by retrieving tickets using license plate numbers or entry date, especially in cases where the camera service did not automatically detect the vehicle.
- **Barrier Control:** The cashier can open or close barriers, depending on the permissions assigned to their role.
- **Article Management:** The system allows the application of special articles such as free tickets, police exemptions, or lost tickets. These articles can modify the payment amount by applying discounts or special rules.
- **Transaction Management:** The cashier can view the list of transactions and access details of each transaction.
- **Receipt Printing:** The system allows printing receipts for completed transactions through the printer service.
- **Shift Closure:** The cashier can close the current shift after completing all operations.

2.2.2 Backoffice Requirements

The backoffice platform is designed to provide administrators with full control over the system configuration and management of operational entities.

The main functional requirements of the backoffice system are as follows:

- **User Management:** The administrator can create, update, and manage user accounts within the system.
- **Role Management:** The administrator can define and manage roles, as well as assign permissions to control access to different functionalities.
- **Cashier Management:** The system allows the management of cashier profiles and their association with POS terminals.
- **Car Park Management:** The administrator can manage car park entities and their configurations.
- **POS Terminal Management:** The system provides functionalities to configure and manage POS terminals deployed in different environments.
- **Article Management:** The administrator can manage articles by defining special rules for them and then assigning them to POS terminals.
- **Camera Management:** The administrator can manage cameras by specifying their IP addresses and then assigning each one to a POS terminal.
- **Barrier Management:** The system allows the management of barriers and their association with POS terminals.
- **Audit and Monitoring:** The administrator can access system logs and audit information to monitor activities and ensure traceability of operations.

2.2.3 Integration Requirements

The proposed system relies on multiple integrations with external systems and services to ensure complete and efficient parking operations. These integrations are designed using a combination of synchronous and asynchronous communication mechanisms.

The system uses HTTP protocols for synchronous communication between services when immediate responses are required. In addition, Server-Sent Events (SSE) are used to enable real-time communication from the backend to the frontend. This mechanism is particularly useful for handling events such as incoming vehicle detection or receiving status updates from external services like the printer service through heartbeat signals.

For asynchronous communication, the system relies on a message broker based on NATS, enabling event-driven interactions between different components. This approach allows decoupling services and ensures better scalability and flexibility in the system architecture.

The main external systems and services integrated into the platform include:

- **Parking Management System (PMS):** Used for data synchronization and managing parking-related information such as vehicle presence, shifts and transactions.
- **Printer Service:** Responsible for handling receipt and report printing operations, with status monitoring through real-time communication.
- **Barrier Service:** Manages the control of entry and exit barriers within the parking infrastructure.
- **Camera Service:** Detects vehicles and triggers events such as payment initiation through system endpoints.
- **Tariff Service:** Handles pricing logic and calculation of parking fees.

This hybrid communication architecture, combining real-time updates, event-driven messaging, and synchronous requests, ensures that the system remains responsive, reliable, and adaptable to complex parking environments.

2.3 Technology Stack and Technical Choices

2.3.1 Backend Stack

2.3.1.1 Go (Golang)

Go is an open-source programming language supported by Google and designed for building simple, secure, and scalable systems [1]. It was chosen for the backend because it offers strong performance, built-in concurrency support, and a clean syntax that fits well with service-oriented development.

2.3.1.2 Gin

Gin is a high-performance HTTP web framework written in Go [2]. It is used in this project to build RESTful APIs with routing, middleware support, and efficient request handling.

2.3.1.3 Bun ORM

Bun is a SQL-first ORM and database client for Go [3]. It was selected for database access because it keeps SQL readable while reducing boilerplate and remaining efficient with PostgreSQL.

2.3.1.4 Zerolog

Zerolog is a fast JSON logger for Go designed with minimal allocations and strong performance in mind [4]. It is used to provide structured logs during the execution of the backend services.

2.3.1.5 JWT

JSON Web Token (JWT) is a compact and secure standard for representing claims between two parties [5]. In this project, JWT is used for authentication and authorization between the frontend applications and the backend APIs.

2.3.1.6 Swaggo

Swaggo is used to generate Swagger-based REST API documentation directly from Go annotations [6]. It helps keep the backend documentation synchronized with the code and improves API readability.

2.3.1.7 PostgreSQL

PostgreSQL is a powerful open-source relational database system that extends SQL with advanced features for reliable and scalable data storage [7]. It is used as the main database engine for both the POS and the backoffice systems.

2.3.1.8 NATS

NATS is a lightweight, high-performance messaging system designed for distributed systems and microservice communication [8]. It is used in this project for event-driven communication and

synchronization between components.

2.3.1.9 Docker

Docker is a containerization platform used to package applications and their dependencies into isolated containers [9]. It is used to simplify local development, deployment, and service isolation.

2.3.2 Frontend Stack

2.3.2.1 React

React is a library for building user interfaces through reusable components [10]. It is used for both the POS and the backoffice frontends because it provides a flexible and component-based development model.

2.3.2.2 TypeScript

TypeScript is a strongly typed programming language built on top of JavaScript [11]. It improves code reliability and developer productivity by adding type safety to the frontend codebase.

2.3.2.3 Tailwind CSS

Tailwind CSS is a utility-first CSS framework for rapidly building modern interfaces [12]. It is used to style the frontend applications through small and reusable utility classes.

2.3.2.4 shadcn/ui

shadcn/ui provides a collection of customizable and accessible UI components for React applications [13]. It is used to build a clean and consistent user interface across the project.

2.3.2.5 Zod

Zod is a TypeScript-first validation library used to define schemas and validate data in a type-safe way [14]. It is used to validate forms and ensure that input data respects the expected structure.

2.3.2.6 Zustand

Zustand is a small, fast, and scalable state management solution for React [15]. It is used to store and share client-side state in a simple and lightweight way.

2.3.2.7 TanStack Router

TanStack Router is a type-safe router for React applications [16]. It is used to manage navigation in the frontoffice applications with a strong TypeScript integration.

2.3.2.8 TanStack Query

TanStack Query is a data-fetching and asynchronous state management library [17]. It is used to fetch, cache, and synchronize server data with the frontend applications.

2.4 Non-Functional Requirements

In addition to functional requirements, the system must satisfy several non-functional requirements to ensure reliability, performance, and usability.

Security: The system must ensure secure access to its functionalities. Authentication is handled using JSON Web Tokens (JWT), allowing controlled and protected access to both the POS and backoffice systems.

Performance: The system must provide fast and efficient responses during operations. The use of Golang with the Gin framework, combined with NATS for messaging, ensures high performance and low latency in both synchronous and asynchronous communications.

Scalability: The architecture must support system growth and increased workload. The use of Golang and a decoupled, service-oriented design allows the system to scale efficiently as the number of POS terminals and users increases.

Availability: The system must remain operational even in case of partial failures. The POS system is designed to operate in offline mode when the connection with external systems is lost, allowing transactions to continue. Once the connection is restored, synchronization is handled through the message broker.

Maintainability: The system must be easy to maintain and evolve. A structured architecture and separation of concerns allow developers to update and extend the system with reduced complexity.

Usability: The system must provide an intuitive and user-friendly interface, particularly for cashiers using the POS. This ensures efficient operation and reduces the learning curve for end users.

2.5 Use Case Modeling

Use case modeling is used to describe the interactions between system actors and the different functionalities provided by the platform. In this project, two main subsystems are identified: the Point of Sale (POS) system and the backoffice system. Each subsystem has its own actors and use cases.

2.5.1 POS Use Case Diagram

The POS system is mainly used by the cashier, who interacts with the system to manage daily parking operations such as payments, shifts, and transactions. The system also communicates with external services such as the PMS, printer service, camera service, and barrier service.

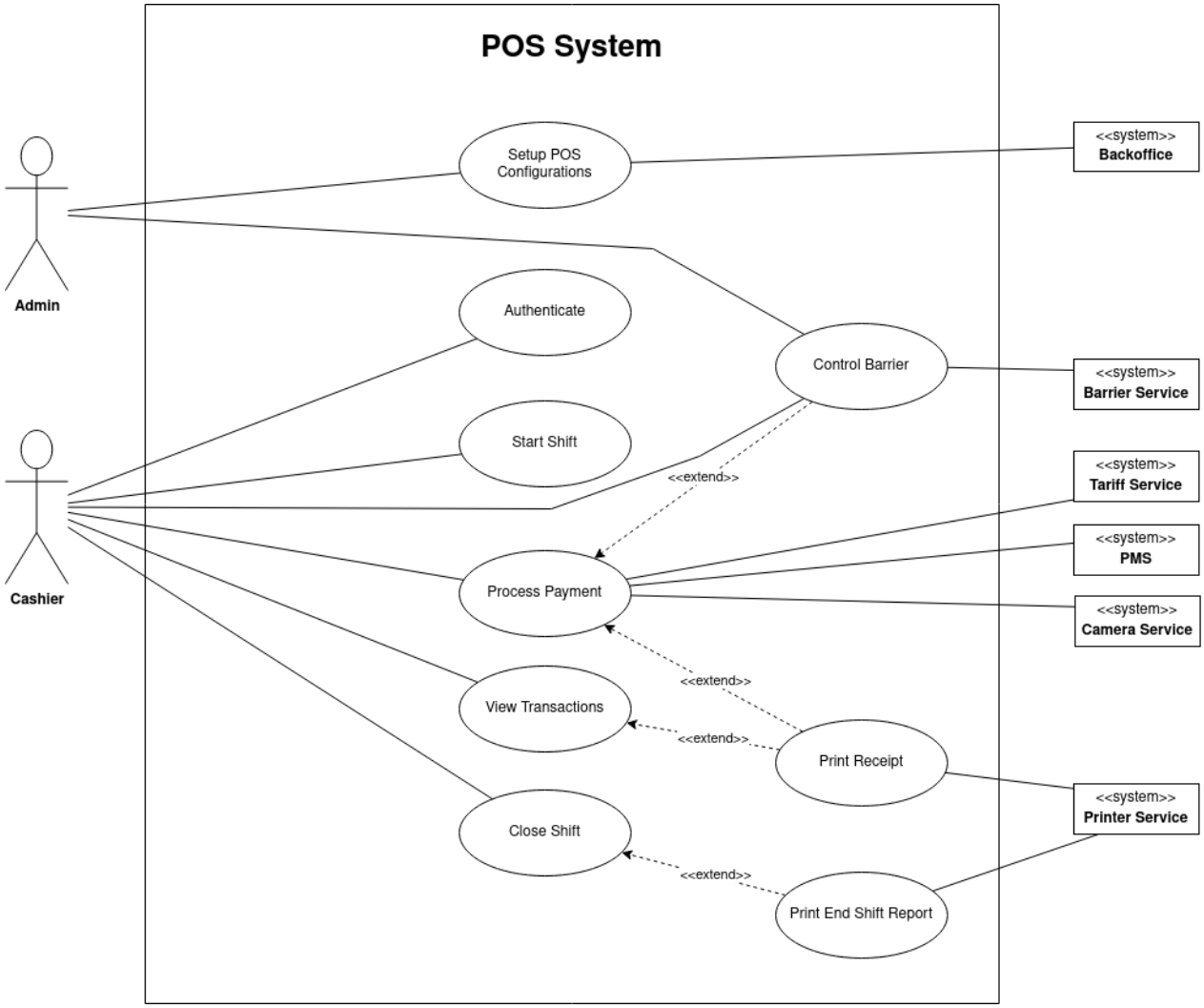

Figure 2.1: POS Use Case Diagram

2.5.2 Backoffice Use Case Diagram

The backoffice system is used by administrators to configure and manage the system. It provides functionalities for managing users, roles, and system entities, as well as monitoring system activity.

Figure 2.2: Backoffice Use Case Diagram

2.6 Scenario Description

This section describes the main interaction scenarios of the system in order to illustrate how different components and actors collaborate during real operations.

2.6.1 Authentication and Shift Management Scenario

The authentication process begins when the cashier accesses the POS system and enters their credentials. If the authentication is successful, the system verifies whether the cashier already has an open shift.

If no shift is currently active, the cashier is prompted to start a new shift by providing an initial amount. Otherwise, a modal dialog is displayed, allowing the cashier to either continue the existing shift or close it before starting a new one.

Once the shift is active, the cashier is redirected to the main POS interface, where they can begin performing daily operations such as processing payments and managing transactions.

At the end of the working session, the cashier can close the shift. Upon closure, a shift report is automatically generated and printed, summarizing all transactions performed during that shift.

2.6.2 Payment Processing Scenario

The payment process is initiated when a vehicle is detected at the exit. The camera service sends a request to the POS backend through a dedicated endpoint. Upon receiving this request, the backend communicates with the frontend using Server-Sent Events (SSE) to display the vehicle and billing information in real time.

The displayed information includes vehicle identification details and the calculated parking fee, which is obtained from the tariff service. If the automatic detection does not occur, the cashier can manually search for the ticket using the license plate number or entry date.

The cashier then proceeds to process the payment using either cash or credit card. Once the payment is successfully completed, the system can automatically trigger the opening of the exit barrier if this feature is enabled. Otherwise, the cashier or an administrator can manually open the barrier.

In parallel, a receipt is generated for the transaction. If automatic printing is enabled, the receipt is printed immediately through the printer service. Otherwise, the cashier can manually trigger the printing operation.

This scenario highlights the integration between different system components and external

services, ensuring a smooth and efficient parking exit process.

Conclusion

This chapter presented the analysis and specification of the system requirements, including the identification of actors, functional and non-functional requirements, integration requirements, and the technology stack selected for implementation. It also described the main use cases and interaction scenarios. The next chapter will focus on the system design.

Chapter 3

System Design

Plan

Introduction

This chapter presents the system design of the proposed solution. It introduces the high-level system architecture and details the functional decomposition of the system. It also covers the event-driven communication design that governs asynchronous flows between system components, the behavioral sequence diagrams describing the main operational scenarios, and the data modeling used to structure the system data.

3.1 High-Level System Architecture

The high-level architecture of the proposed system provides a global view of how the different components interact to support parking operations, particularly the processing of exiting vehicles. This architecture focuses on the main systems involved without going into technical implementation details, in order to highlight the overall structure and responsibilities of each component.

The system is organized around several core components, each playing a specific role:

- **POS System:** Used by the cashier to manage daily operations, including starting shifts and processing payments for exiting vehicles.
- **Backoffice System:** Provides configuration and administration functionalities such as managing POS terminals, car parks, users, roles, and related system parameters.
- **PMS (Parking Management System):** Acts as the central system for managing parking data. It stores presence tickets (vehicle entries) and synchronizes transaction data generated by the POS system.
- **Camera System:** Interacts with camera hardware to detect vehicles. It sends vehicle information to the POS system when a car reaches the exit.
- **Printer System:** Responsible for printing receipts and shift reports. It receives formatted data from the POS system and communicates with the physical printer.
- **Barrier System:** Controls the physical access barrier by opening or closing it based on commands received from the POS or Backoffice systems.

The main operational flow of the system starts when a vehicle enters the parking area and is

registered by the PMS. When the vehicle reaches the exit, the camera system detects it and sends the corresponding information to the POS system. The cashier then processes the payment, after which the barrier is opened to allow the vehicle to leave. During this process, a receipt can be generated and printed if required.

The Backoffice system plays an essential role in configuring and managing the overall environment. It allows administrators to manage multiple POS terminals and ensure that all system components are correctly configured and synchronized.

At this level of abstraction, the communication between systems is represented as interactions through well-defined interfaces. The architecture emphasizes a modular approach, where each component operates independently while collaborating with others. This design improves system flexibility, supports real-time processing of parking operations, and facilitates future extensions or integrations.

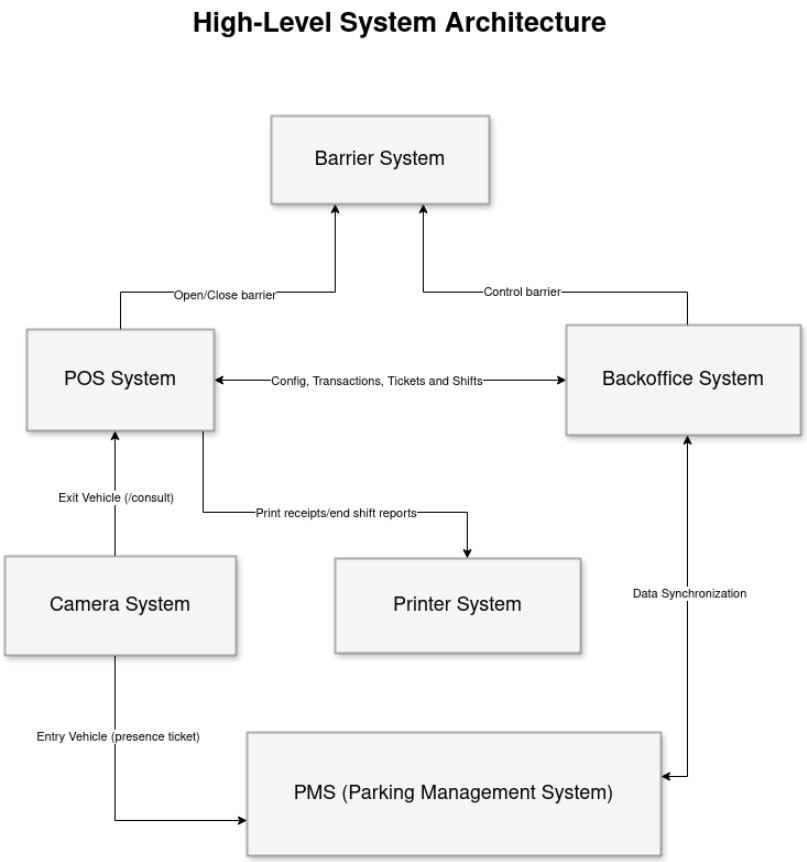

Figure 3.1: High Level System Architecture

3.2 Functional Decomposition

Functional decomposition consists of breaking down the system into smaller functional units in order to better understand its structure and responsibilities. In this project, the system is divided into three main parts: the POS system, the backoffice system, and its interactions with external services.

3.2.1 POS System Functions

The Point of Sale (POS) system is responsible for handling real-time parking operations at exit points. Its functionalities are organized into several main groups.

Authentication and Session Management

- Cashier login
- Pause shift (temporary logout without closing the shift)
- End shift (closing the shift and logging out)

Shift Management

- Start a new shift with an initial amount
- Continue an existing open shift
- Close shift and generate end-of-shift report

Ticket Search

- Search for tickets manually using license plate number or entry time range
- Display vehicle and parking information

Payment Processing

- Process payments (cash or card)
- Automatically retrieve ticket information when triggered by the camera system
- Calculate parking fees via external tariff service
- Print receipt (automatic or manual)

- Open barrier automatically or manually after payment

Transaction Management

- View transactions history
- Search for specific transactions
- Reprint receipts when needed

Hardware Interaction

- Communicate with printer service for receipt and report printing
- Control barrier system (open/close)

3.2.2 Backoffice System Functions

The backoffice system is dedicated to configuration and administrative management. It allows administrators to control and monitor the overall system behavior.

User and Role Management

- Manage users (create, update, delete, list)
- Manage roles and access rights

Configuration Management

- Manage cashiers
- Manage carparks
- Manage POS terminals
- Manage articles (special pricing rules)
- Manage barriers
- Manage cameras

Monitoring

- View audit logs
- Track system activities and operations

3.2.3 External System Interactions

The system interacts with several external services to ensure proper operation of parking processes.

- **PMS (Parking Management System):** Provides presence tickets (entry data), manages shifts for cashiers, receives transaction data for synchronization, and manages cashier shifts, including online validation and offline (standalone) operation with subsequent synchronization.
- **Camera System:** Detects exiting vehicles and triggers the payment process via the POS system.
- **Printer Service:** Handles printing of receipts and shift reports based on requests from the POS system.
- **Barrier System:** Controls physical gate operations (opening and closing) after payment validation.
- **Tariff Service:** Calculates parking fees based on duration and pricing rules.

3.3 Event-Driven Communication Design

The system is designed around two communication paradigms. Synchronous communication handles operations that require an immediate response, such as API calls between the frontend and backend. Asynchronous communication handles operations where decoupling is more important than immediacy, particularly events that need to propagate across multiple components without creating dependencies between them.

Three main asynchronous flows are identified at the design level.

3.3.1 Shift Lifecycle Synchronization

When a cashier starts or closes a shift, the operation is recorded locally on the POS terminal first. In parallel, the event is dispatched asynchronously toward the central system for synchronization. This

means the cashier is immediately redirected to the POS interface without waiting for any external acknowledgment, while the synchronization happens independently in the background.

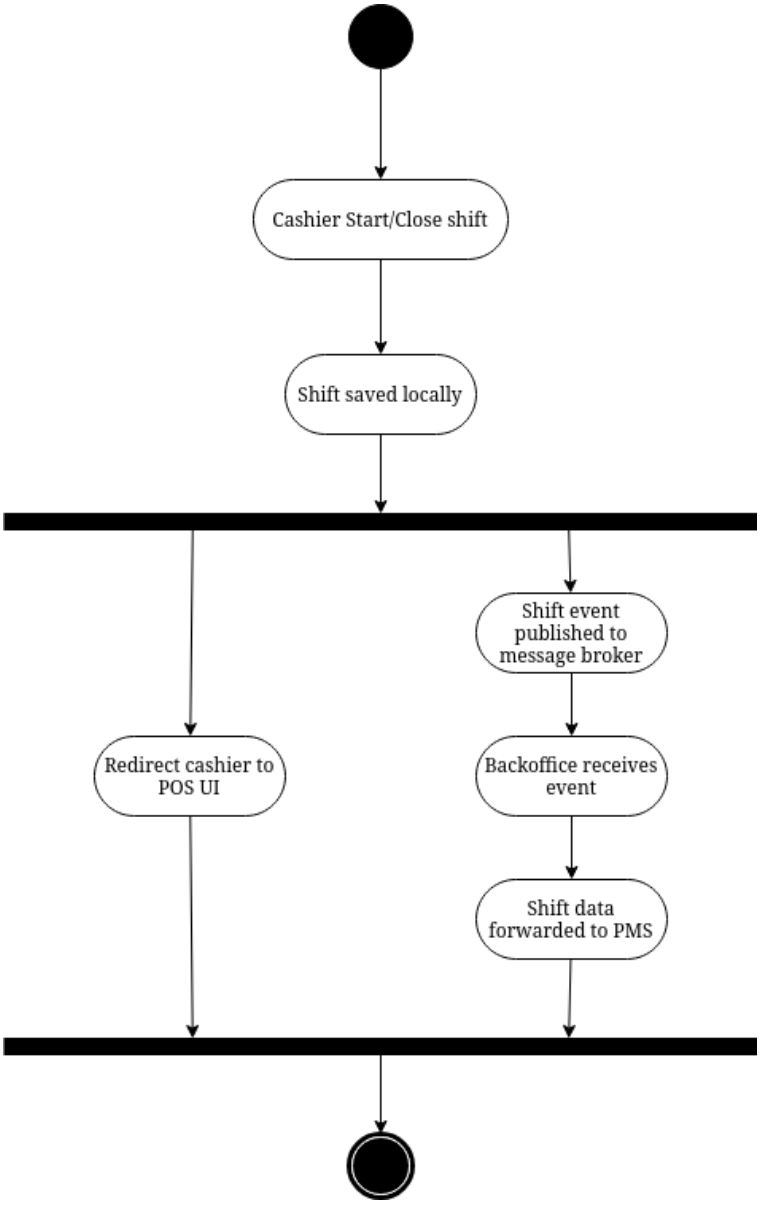

Figure 3.2: Shift lifecycle synchronization activity diagram

3.3.2 Payment and Ticket Exit Propagation

After a payment is completed and the transaction is saved locally, the barrier opens and the receipt is printed. At the same time, two independent events are dispatched in parallel. The first carries the transaction data toward the central system for record-keeping. The second notifies all other POS terminals in the facility that the corresponding vehicle ticket is no longer active. This ensures

consistency across terminals without any direct coupling between them.

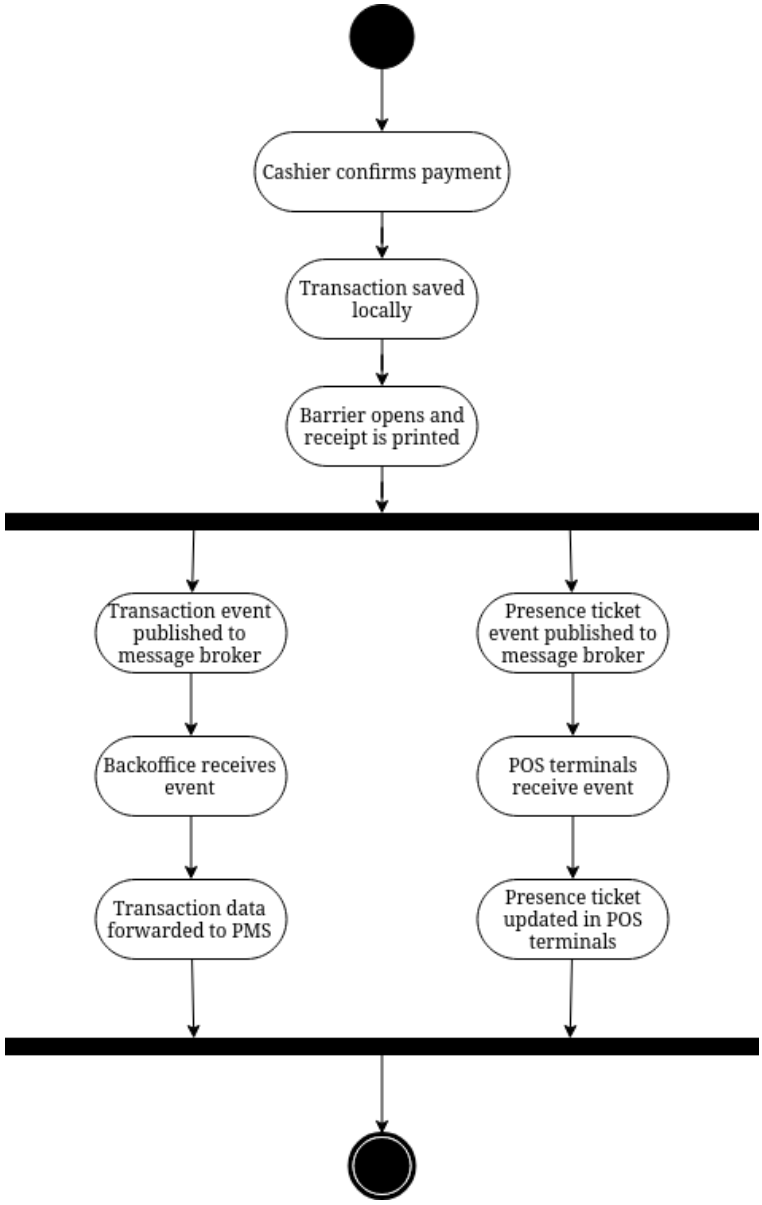

Figure 3.3: Payment and ticket exit propagation activity diagram

3.3.3 Configuration Propagation

When an administrator updates a POS terminal configuration from the backoffice, the change is saved locally in the backoffice first. In parallel, a configuration event is dispatched and addressed to the specific terminal. The terminal receives it and applies the update immediately, without requiring any direct connection between the backoffice and the terminal at that moment.

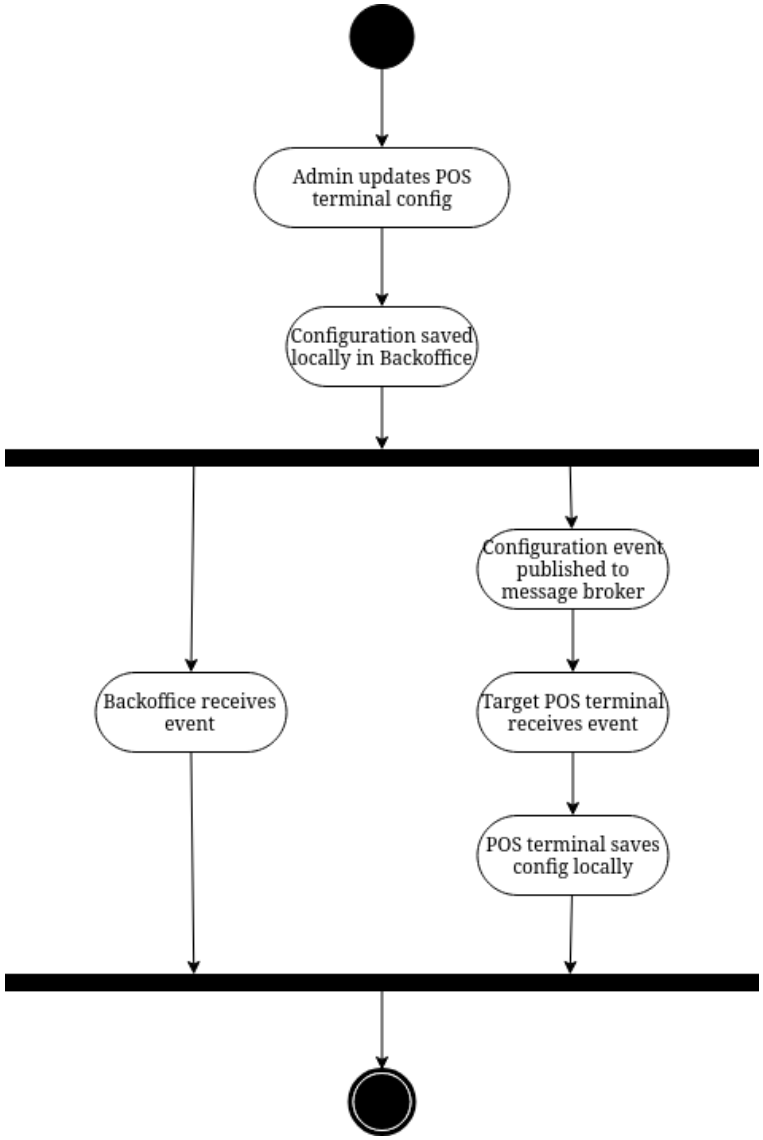

Figure 3.4: Configuration propagation activity diagram

3.4 Behavioral Design

This section presents the main sequence diagrams that describe the dynamic behavior of the system during key runtime scenarios. These diagrams illustrate the chronological flow of interactions between system actors and the different components of the application, including user interfaces, controllers, and core entities.

3.4.1 Setup POS Configurations

Tableau 3.1: Textual description of the use case «Setup POS Configurations»

Use Case	Setup POS Configurations
Actor	Admin
Pre-condition	The POS terminal has not yet been configured.
Post-condition	The POS configuration is saved and the admin is redirected to the cashier login page.
Normal Scenario	1. The admin opens the setup page, 2. The system displays the configuration form, 3. The admin enters the backoffice base URL and the POS token, 4. The admin clicks validate, 5. The system verifies the provided credentials against the backoffice, 6. The configuration is saved, 7. The admin is redirected to the cashier login page.
Alternate Scenarios	3. The admin clicks test connection before validating, 3.1. The system sends a test request to the backoffice, 3.2. If successful, a success message is displayed, 3.3. If it fails, an error message is shown and the admin can correct the inputs. 5. The credentials are invalid, 5.1. The system displays a validation error message, 5.2. Return to step 3 of the normal scenario.

This sequence diagram illustrates the initial configuration of a POS terminal by an administrator. The admin provides the backoffice base URL and the POS token through a setup form. The admin can either test the connection to verify the credentials or directly validate and save the configuration. On successful validation, the settings are persisted and the admin is redirected to the cashier login

page.

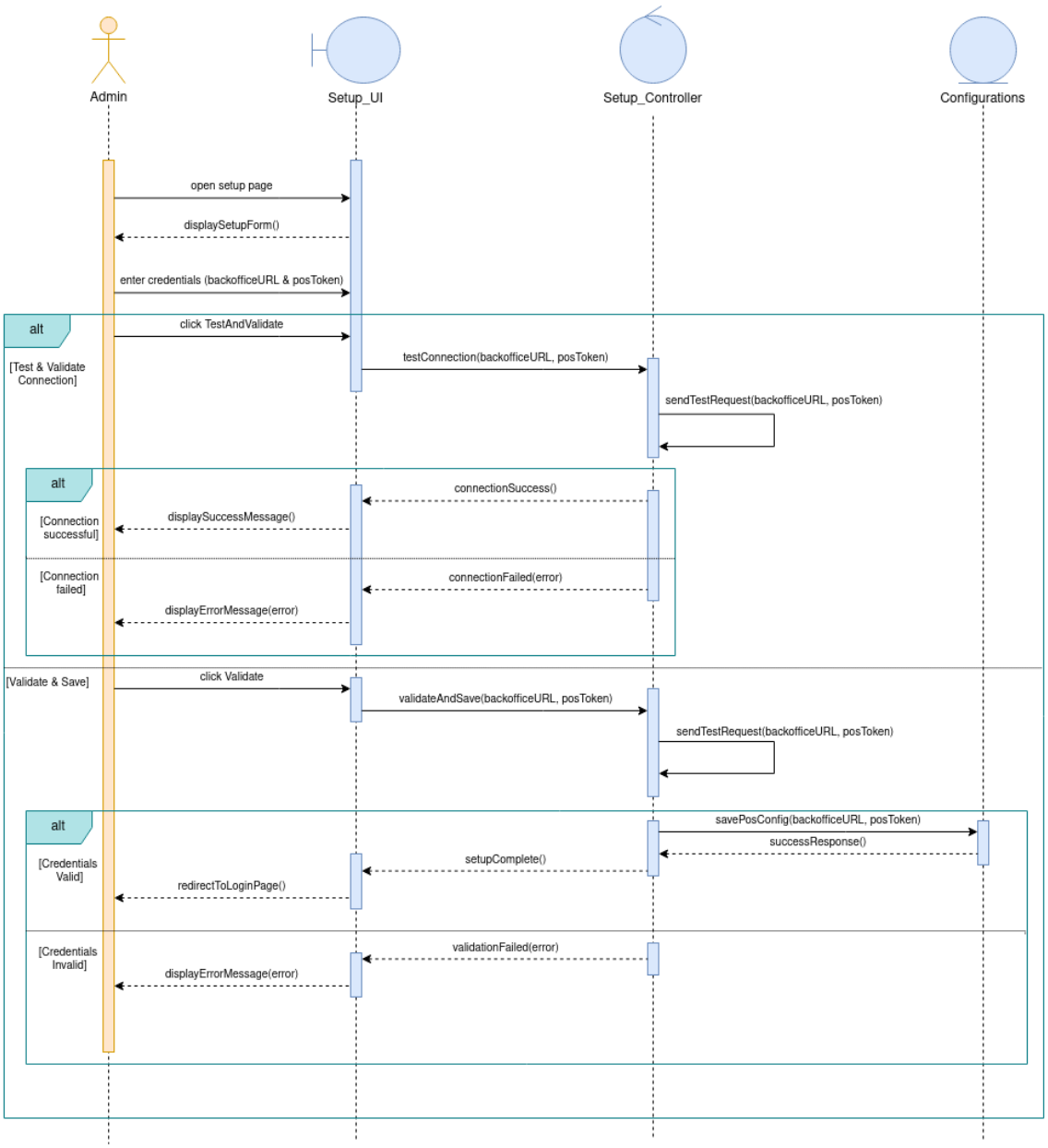

Figure 3.5: Setup POS configurations sequence diagram

3.4.2 Authenticate and Start Shift

Tableau 3.2: Textual description of the use case «Authenticate and Start Shift»

Use Case	Authenticate and Start Shift
Actor	Cashier
Pre-condition	The cashier has valid credentials and access to the POS terminal.
Post-condition	The cashier is authenticated and an active shift is open, granting access to the main POS interface.
Normal Scenario	1. The cashier opens the login page, 2. The system displays the login form, 3. The cashier enters their username and password, 4. The system validates the credentials, 5. An authentication token is generated and returned, 6. The system checks whether an open shift exists for the cashier, 7. No open shift is found, the cashier enters a starting amount, 8. The system creates a new shift, 9. The cashier is redirected to the main POS interface.
Alternate Scenarios	4. The credentials are invalid, 4.1. The system displays an error message, 4.2. Return to step 3 of the normal scenario. 6. An open shift already exists, 6.1. The system prompts the cashier to continue or start a new shift, 6.2. If the cashier chooses to continue, access to the POS interface is granted directly, 6.3. If the cashier chooses a new shift, the current shift is closed and a new one is created with a specified starting amount.

This sequence diagram covers the login and shift initialization flow in the POS system. The cashier enters their credentials, which are validated by the authentication controller. On success, the system checks whether an open shift already exists for that cashier. If so, the cashier can either continue it or close it and start a new one with a specified starting amount. If no shift is found, a new one is created directly. Authentication failure results in an error message being displayed.

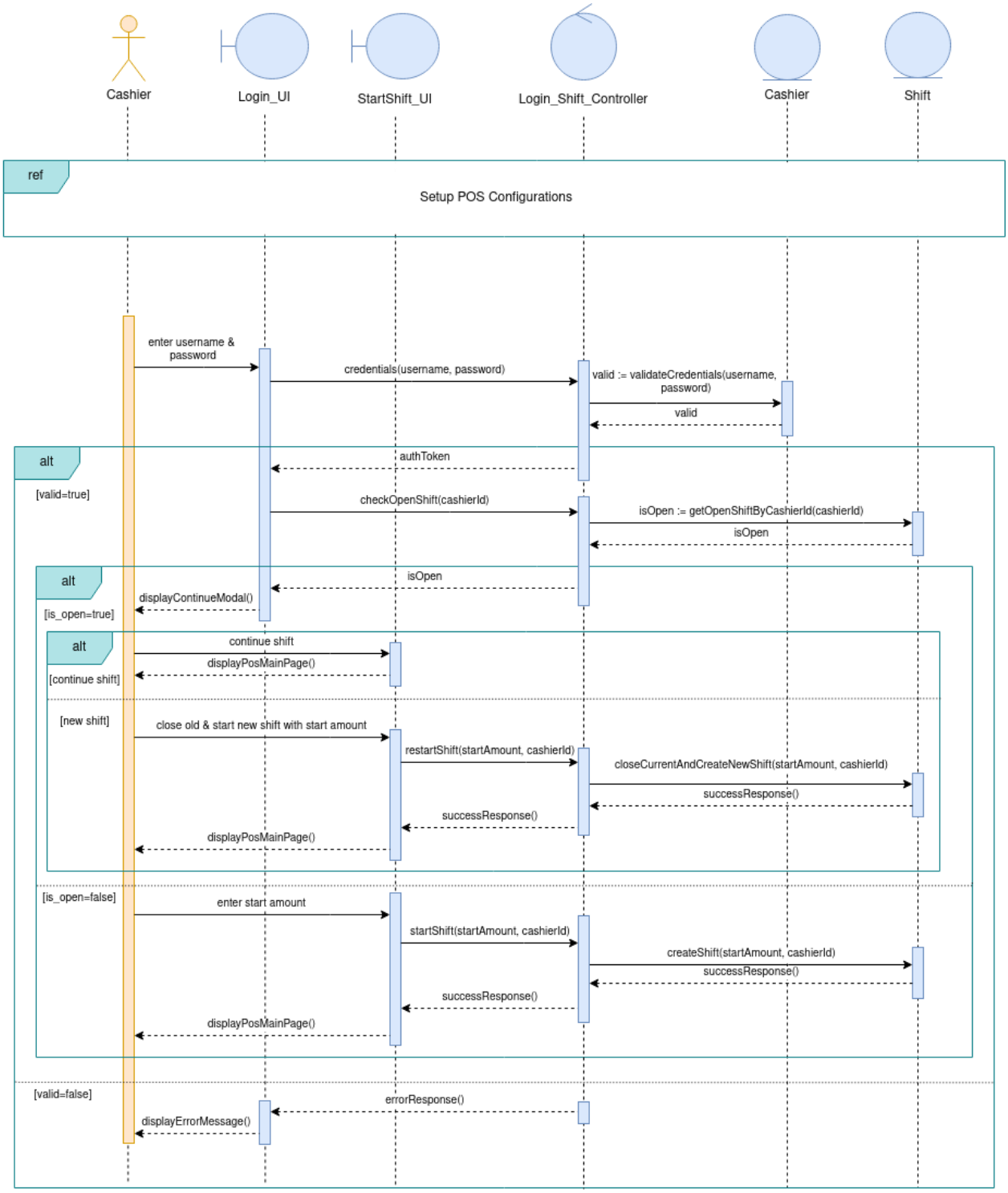

Figure 3.6: Authenticate and Start Shift sequence diagram

3.4.3 Process Payment

Tableau 3.3: Textual description of the use case «Process Payment»

Use Case	Process Payment
Actors	Cashier, Camera
Pre-condition	The cashier has an active shift open and the camera is operational.
Post-condition	The transaction is saved, the receipt is printed, and the barrier is opened.
Normal Scenario	1. The camera detects a vehicle exiting and sends the license plate number, 2. The system retrieves the corresponding presence ticket, 3. A payment object is initialized and displayed to the cashier, 4. The cashier optionally applies one or more articles to adjust the amount, 5. The cashier selects a payment method and confirms the transaction, 6. The system saves the transaction, 7. The receipt is printed and the barrier is opened in parallel.
Alternate Scenarios	2. No presence ticket is found for the license plate, 2.1. The system displays a search modal for manual LPN entry, 2.2. The cashier enters the license plate manually, 2.3. Return to step 2 of the normal scenario. 6. The transaction fails to save, 6.1. The system displays an error message to the cashier.

This sequence diagram describes the payment processing workflow. The camera detects an exiting vehicle and sends its license plate to the POS system, which retrieves the corresponding presence ticket. A payment object is then initialized and displayed to the cashier. The cashier can optionally apply articles to adjust the amount, then selects a payment method and confirms the transaction. On success, the receipt is printed and the barrier is opened in parallel.

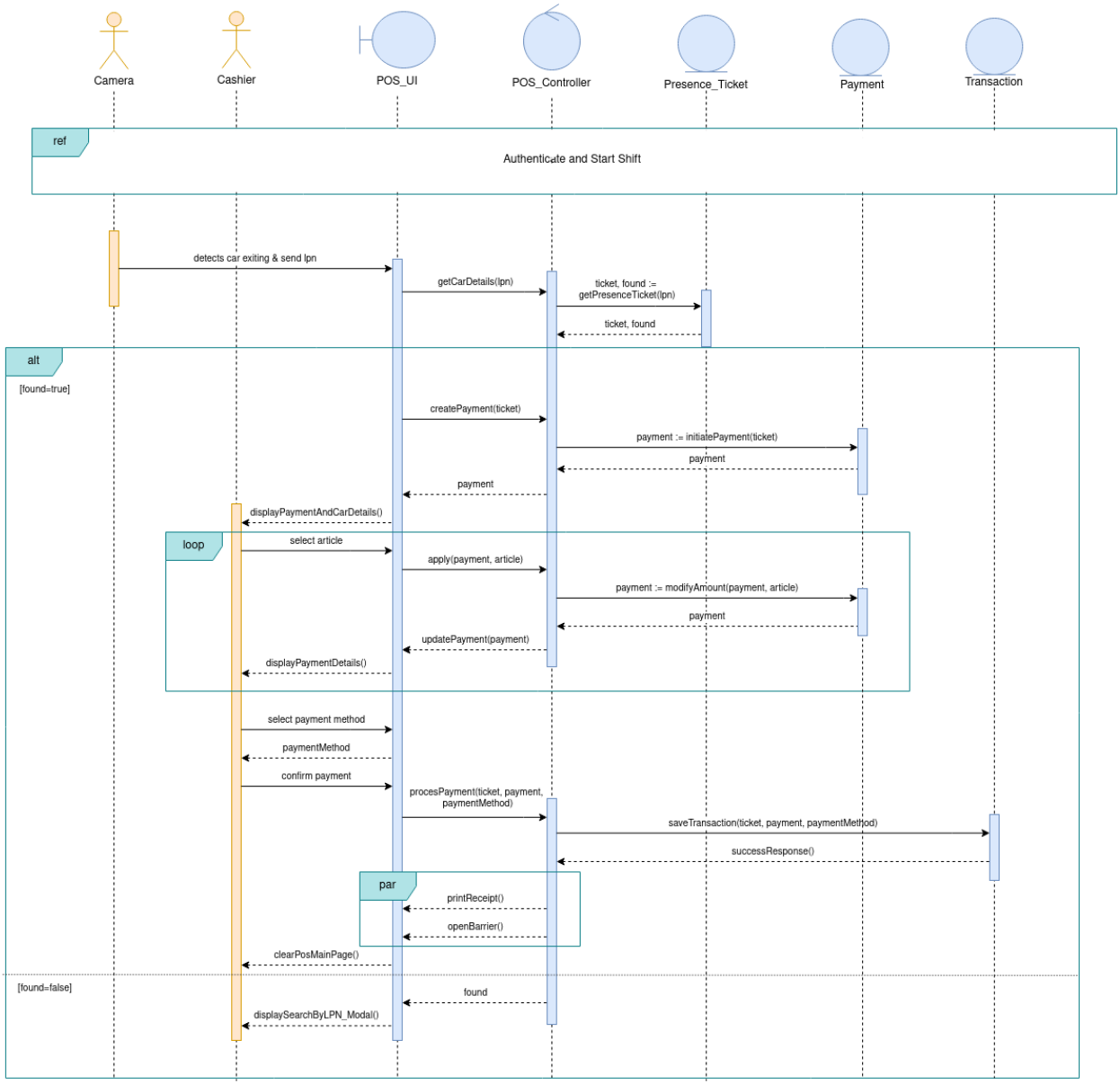

Figure 3.7: Process Payment sequence diagram

3.4.4 Control Barrier

Tableau 3.4: Textual description of the use case «Control Barrier»

Use Case	Control Barrier
Actors	Cashier, Administrator
Pre-condition	The cashier is authenticated and has an active shift.
Post-condition	The barrier action is executed and the event is recorded.
Normal Scenario	1. The cashier clicks the barrier button on the POS interface, 2. The system displays the available barrier actions (Lock, Unlock, Open, Close), 3. The cashier selects Lock or Close, 4. The system executes the action directly and records the event, 5. A success message is displayed to the cashier.
Alternate Scenario	3. The cashier selects Unlock or Open, 3.1. The system checks whether the cashier has the required permission, 3.2. If the cashier has permission, the action is executed and the event is recorded, 3.3. If the cashier does not have permission, an admin validation prompt is displayed, 3.4. The administrator enters the evaluation password, 3.5. If the password is valid, the action is executed and the event is recorded under the admin, 3.6. If the password is invalid, an error message is displayed and the action is cancelled.

This sequence diagram covers the barrier control feature available from the POS interface. The cashier can perform four actions on the barrier: Lock, Unlock, Open, and Close. Lock and Close are executed directly without any permission check. Unlock and Open require verifying whether the cashier has the necessary permission. If the permission is granted, the action proceeds normally.

Otherwise, the system prompts for an administrator evaluation password. If the password is valid, the administrator effectively authorizes the action on behalf of the cashier. Each executed action is recorded as a barrier event in the system.

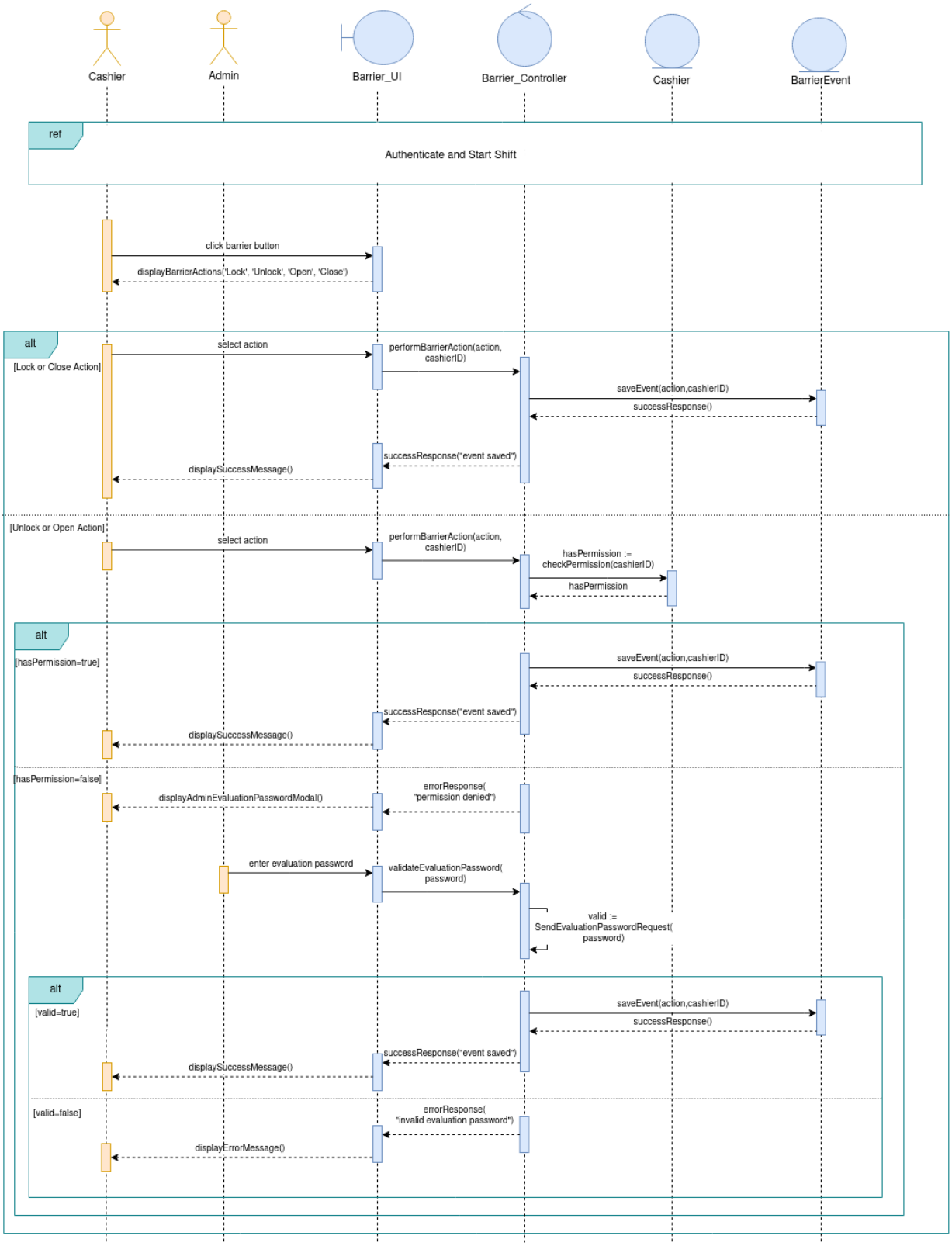

Figure 3.8: Control barrier sequence diagram

3.4.5 View Transactions

Tableau 3.5: Textual description of the use case «View Transactions»

Use Case	View Transactions
Actor	Cashier
Pre-condition	The cashier is authenticated and has an active shift.
Post-condition	The cashier has consulted the desired transaction details or printed the receipt.
Normal Scenario	1. The cashier opens the transactions page, 2. The system loads and displays the paginated list of transactions for the current shift, 3. The cashier browses the list.
Alternate Scenario	3. The cashier searches by license plate number, 3.1. The system filters and returns the matching transactions. 3. The cashier previews a receipt for a specific transaction, 3.1. The system retrieves the transaction details and displays the receipt preview. 3. The cashier prints a receipt, 3.1. The system retrieves the transaction details and sends the print request.

This sequence diagram covers the transaction consultation feature available to the cashier. Upon opening the transactions page, the system loads the paginated list of transactions for the current shift. The cashier can search by license plate number, preview a receipt for a specific transaction, or print it directly. Each action triggers a request to the transaction controller, which queries the Transaction entity accordingly.

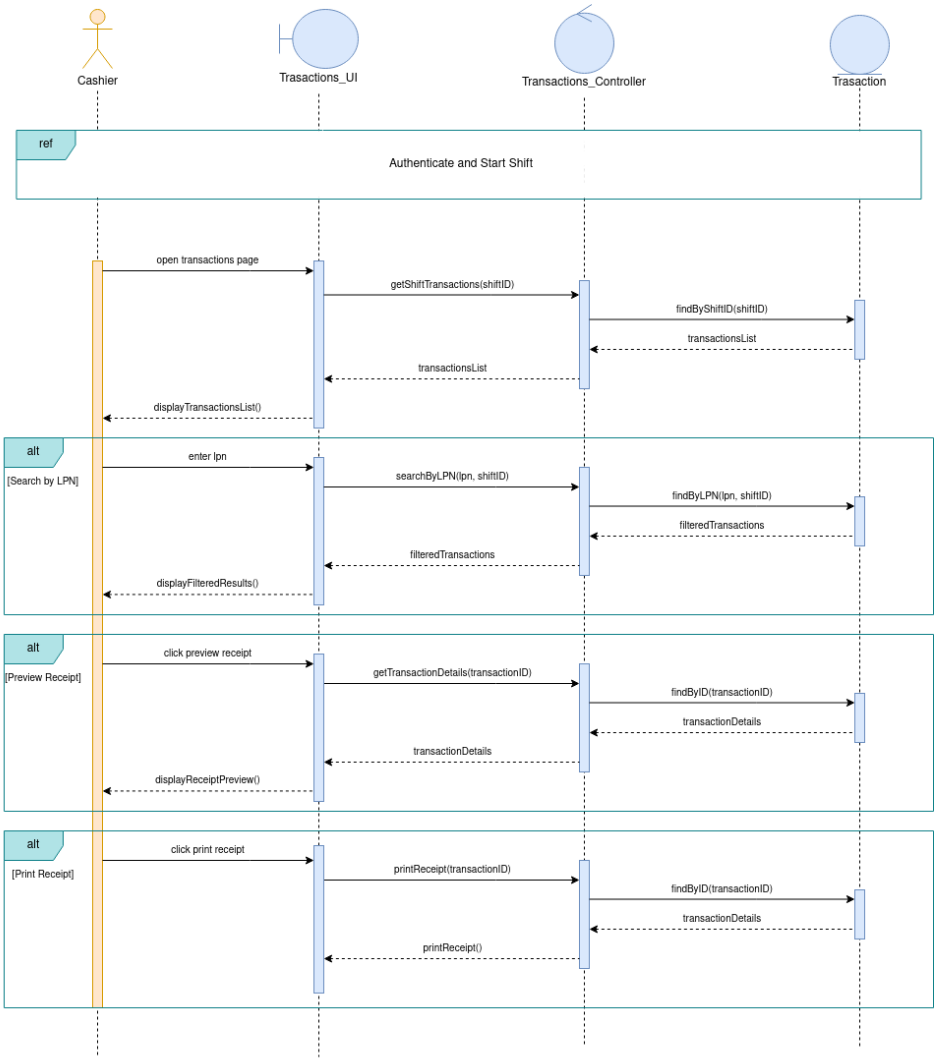

Figure 3.9: View transactions sequence diagram

3.4.6 Close Shift

Tableau 3.6: Textual description of the use case «Close Shift»

Use Case	Close Shift
Actor	Cashier
Pre-condition	The cashier is authenticated and has an active open shift.
Post-condition	The shift is closed, the end shift report is printed, and the cashier is redirected to the login page.
Normal Scenario	1. The cashier selects close shift, 2. The system displays a confirmation prompt, 3. The cashier confirms, 4. The system closes the shift and retrieves the shift summary, 5. The end shift report is printed, 6. The cashier is redirected to the login page.
Alternate Scenario	1. The cashier selects pause shift instead, 1.1. The system redirects the cashier to the login page without closing the shift or contacting the backend.

This sequence diagram describes the shift closing process. The cashier can either pause the shift, which simply redirects to the login page without any backend interaction, or close it entirely. In the latter case, the cashier confirms the action and provides the end amount. The controller then closes the shift, retrieves the shift summary, and triggers the printing of the end shift report and then redirect the cashier to the login page.

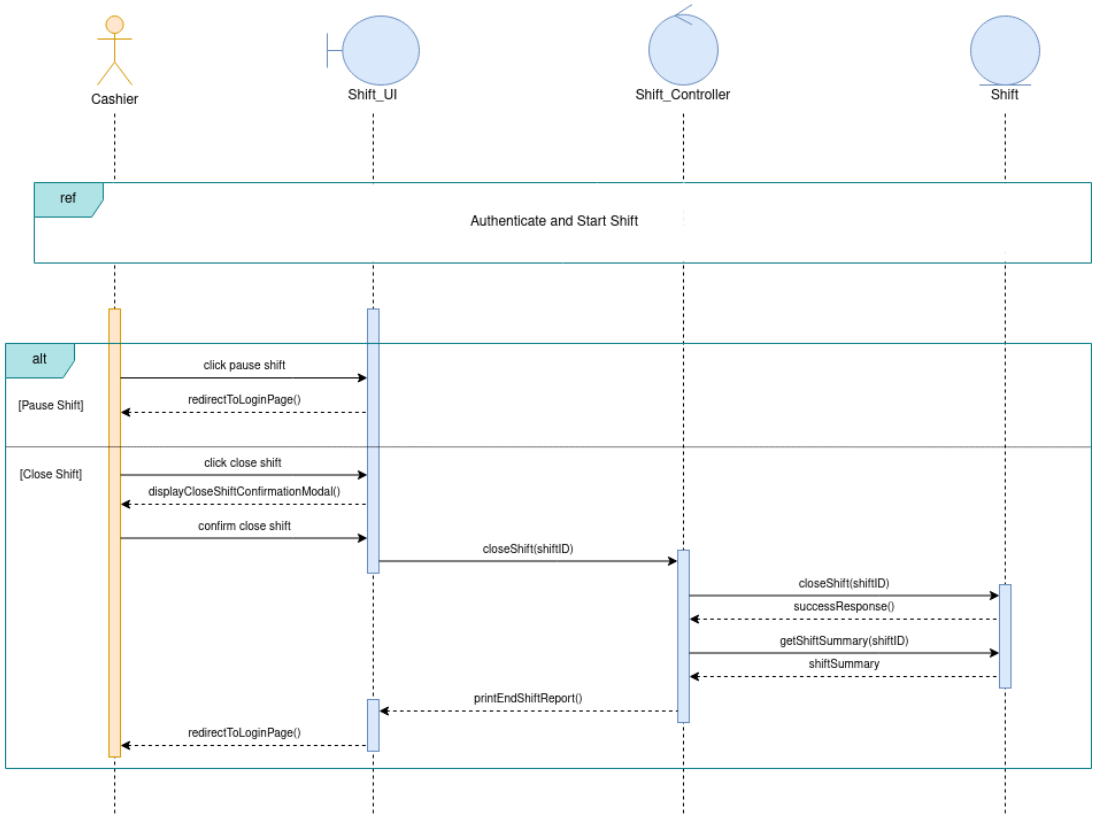

Figure 3.10: Close shift sequence diagram

3.4.7 Authenticate Admin

Tableau 3.7: Textual description of the use case «Authenticate Admin»

Use Case	Authenticate Admin
Actor	Administrator
Pre-condition	The administrator has a valid account in the system.
Post-condition	The administrator is authenticated and redirected to the backoffice dashboard.
Normal Scenario	1. The administrator opens the backoffice login page, 2. The system displays the login form, 3. The administrator enters their email and password, 4. The system looks up the user by email, 5. The system validates the provided password, 6. An authentication token is generated, 7. The administrator is redirected to the dashboard.
Alternate Scenarios	4. No user is found with the provided email, 4.1. The system displays an error message, 4.2. Return to step 3 of the normal scenario. 5. The password is invalid, 5.1. The system displays an error message, 5.2. Return to step 3 of the normal scenario.

This sequence diagram illustrates the administrator login flow for the backoffice system. The admin submits their email and password through the login interface. The controller looks up the user by email and validates the provided password. On success, an authentication token is generated and the admin is redirected to the dashboard. Otherwise, an error message is displayed.

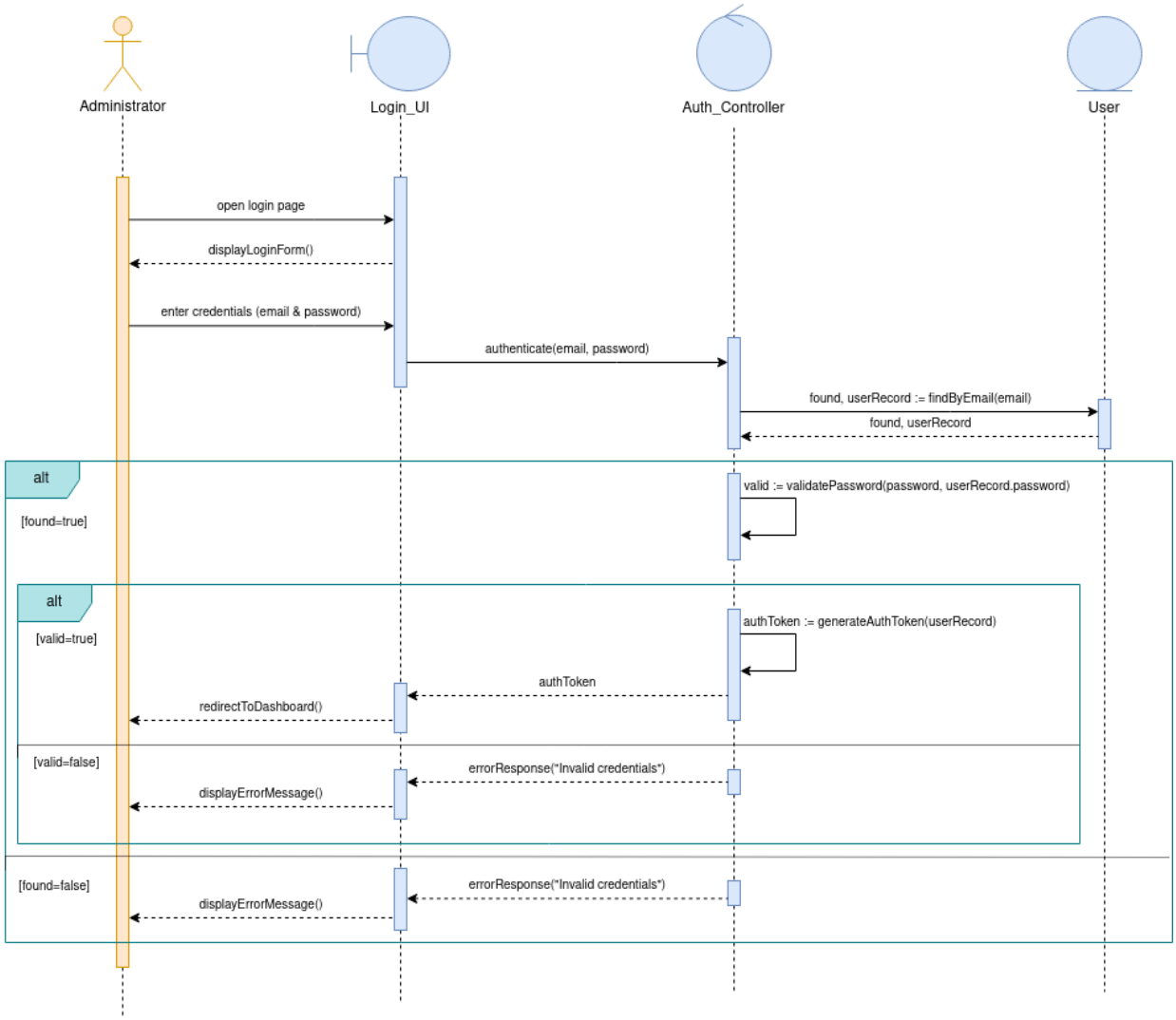

Figure 3.11: Authenticate Admin sequence diagram

3.4.8 Manage Articles

Tableau 3.8: Textual description of the use case «Manage Articles»

Use Case	Manage Articles
Actor	Administrator
Pre-condition	The administrator is authenticated and has the required permissions.
Post-condition	The articles list reflects the changes made by the administrator.
Normal Scenario	1. The administrator opens the articles page, 2. The system loads and displays the paginated list of articles, 3. The administrator selects an action (add, edit, or delete), 4. For add: the administrator fills in the article form and submits it, 5. For edit: the administrator modifies the article details and submits, 6. For delete: the administrator confirms the deletion, 7. The system processes the request and refreshes the articles list.
Alternate Scenarios	4. The submitted form contains invalid data, 4.1. The system displays a validation error message, 4.2. Return to step 4 of the normal scenario. 6. The administrator cancels the deletion, 6.1. The system dismisses the confirmation dialog and no changes are made.

This sequence diagram represents the article management feature in the backoffice, and serves as a representative example for all CRUD-based use cases in the system, including Manage Roles, Users, Carparks, Cashiers, POS Terminals, Barriers, and Cameras, as they all follow the same interaction

pattern. The flow assumes the administrator is already authenticated. Upon opening the articles page, a paginated list is loaded. The administrator can then add a new article by filling in a form, edit an existing one, or delete it after confirmation. Each action is handled by the article controller and reflected in the articles list.

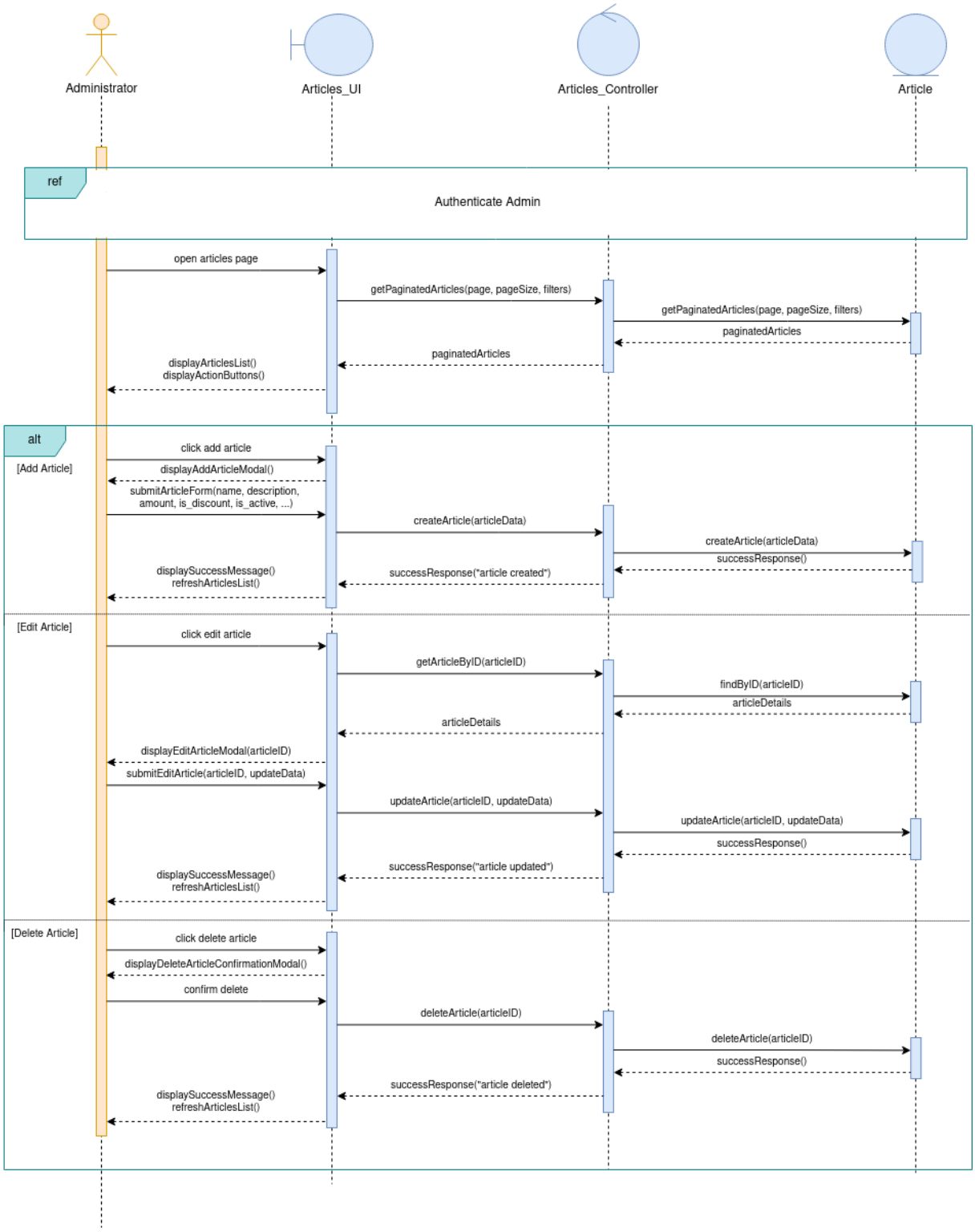

Figure 3.12: Manage articles sequence diagram

3.5 Data Modeling and Class Diagrams

This section presents the main class diagrams used to represent the structure of the system data and the relationships between the main entities of the application. These diagrams help clarify how the POS side and the backoffice side are organized from a modeling point of view.

The POS class diagram illustrates the main entities involved in the operational part of the system. It gives an overview of how the data used by the POS is structured and how the related elements are connected inside the application.

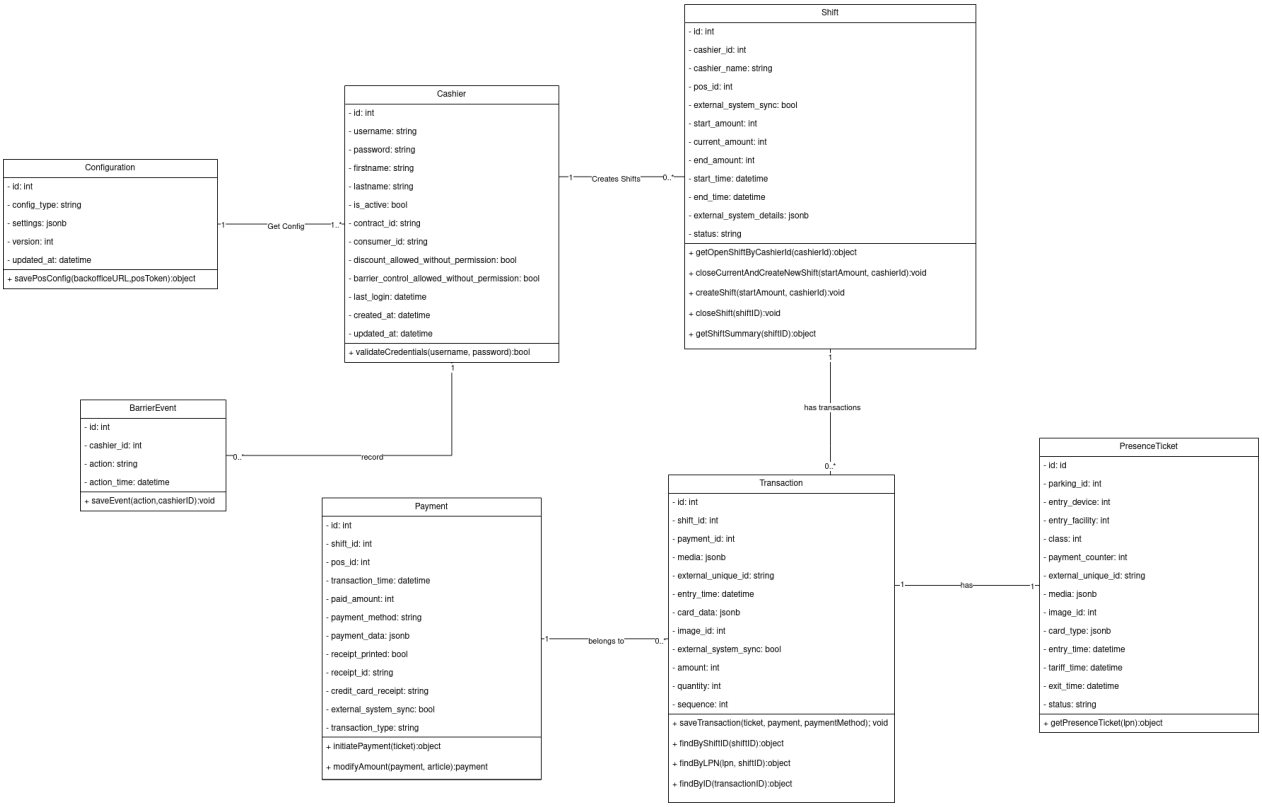

Figure 3.13: POS class diagram

The backoffice class diagram presents the main entities related to the administrative and configuration side of the platform. It highlights the structure used to manage the backoffice data and the links between its principal components.

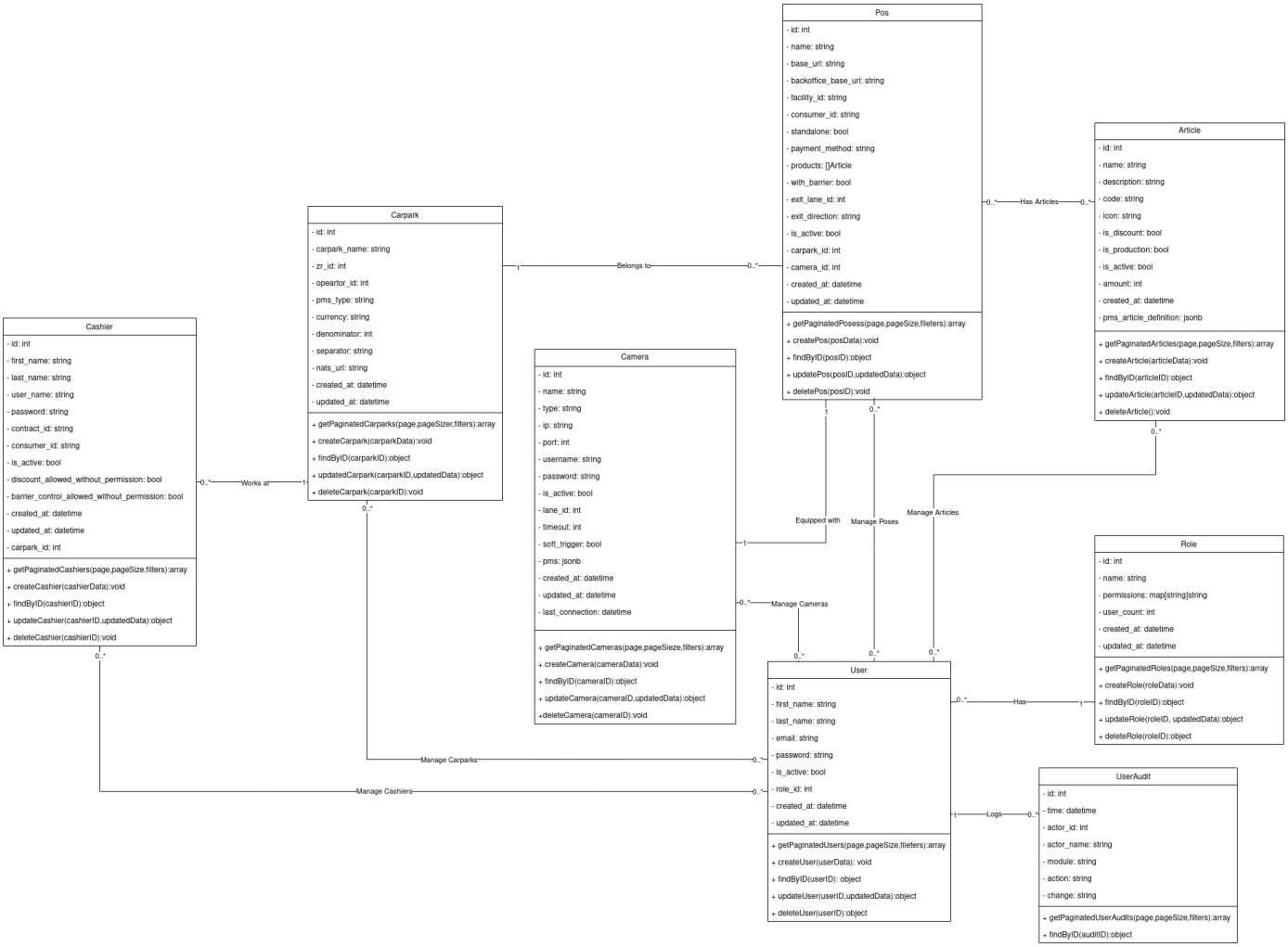

Figure 3.14: Backoffice class diagram

Conclusion

This chapter presented the overall design of the proposed solution, including the high-level system architecture, the functional decomposition of the POS and backoffice platforms, the event-driven communication design, sequence diagrams, and the data modeling approach. These design choices established a modular and scalable foundation adapted to the operational constraints of modern parking systems, while preparing the implementation phase described in the next chapter.

Chapter 4

Implementation and Realization

Plan

Introduction

This chapter presents the practical realization of the proposed solution. It describes the development environment, the implemented system architecture, and the key performance-oriented design decisions. The chapter also covers the integration with external parking services, illustrates the developed user interfaces, and summarizes the testing and validation activities carried out during the internship.

4.1 Development Environment

4.1.1 Development Workstation

The development work was carried out on a personal machine running *EndeavourOS* with *i3wm* as the window manager. This lightweight Linux environment was suitable for daily development tasks and provided a stable working setup for building both the POS and backoffice applications [18], [19].

4.1.2 Containerized Execution Model

All services of the solution are containerized using Docker [9]. Each component is packaged with its own Dockerfile, which makes the execution environment isolated, reproducible, and easier to deploy on different machines. This approach also simplifies local testing and allows the same application behavior to be reproduced in the company environment and, later on, in the client environment.

4.1.3 Deployment and Company Environment

For testing purposes, the backoffice is deployed in the company infrastructure using a GitLab CI/CD workflow [20]. The pipeline builds the Docker image, pushes it to a private Nexus registry [21], and deploys it through Portainer [22]. This setup is used during the internship to validate the deployment process before delivery to the final client environment.

The POS terminals are intended to run on Linux machines at the client side, since the project is designed to support multiple POS installations. During the internship, the same Linux-based deployment style was used in order to stay close to the real execution environment expected on the client infrastructure.

4.1.4 Development and Validation Tools

Several tools were used during the development process to improve productivity and validate the behavior of the system. *Neovim* was the main text editor [23], while *Zed* was also used occasionally. *Postman* [24] and *cURL* [25] were used to test API endpoints, and *DBeaver* [26] together with *psql* were used to inspect and query the PostgreSQL databases locally.

4.2 Implemented System Architecture

The solution is built around a modular architecture where each component has a focused responsibility. The POS system handles cashier operations and parking exit workflows. The backoffice manages configuration, administration, and acts as a relay between POS terminals and the central parking system. Communication follows a hybrid model: synchronous HTTP for immediate responses, and asynchronous messaging through NATS for event-driven interactions such as shift synchronization, transaction propagation, and configuration updates.

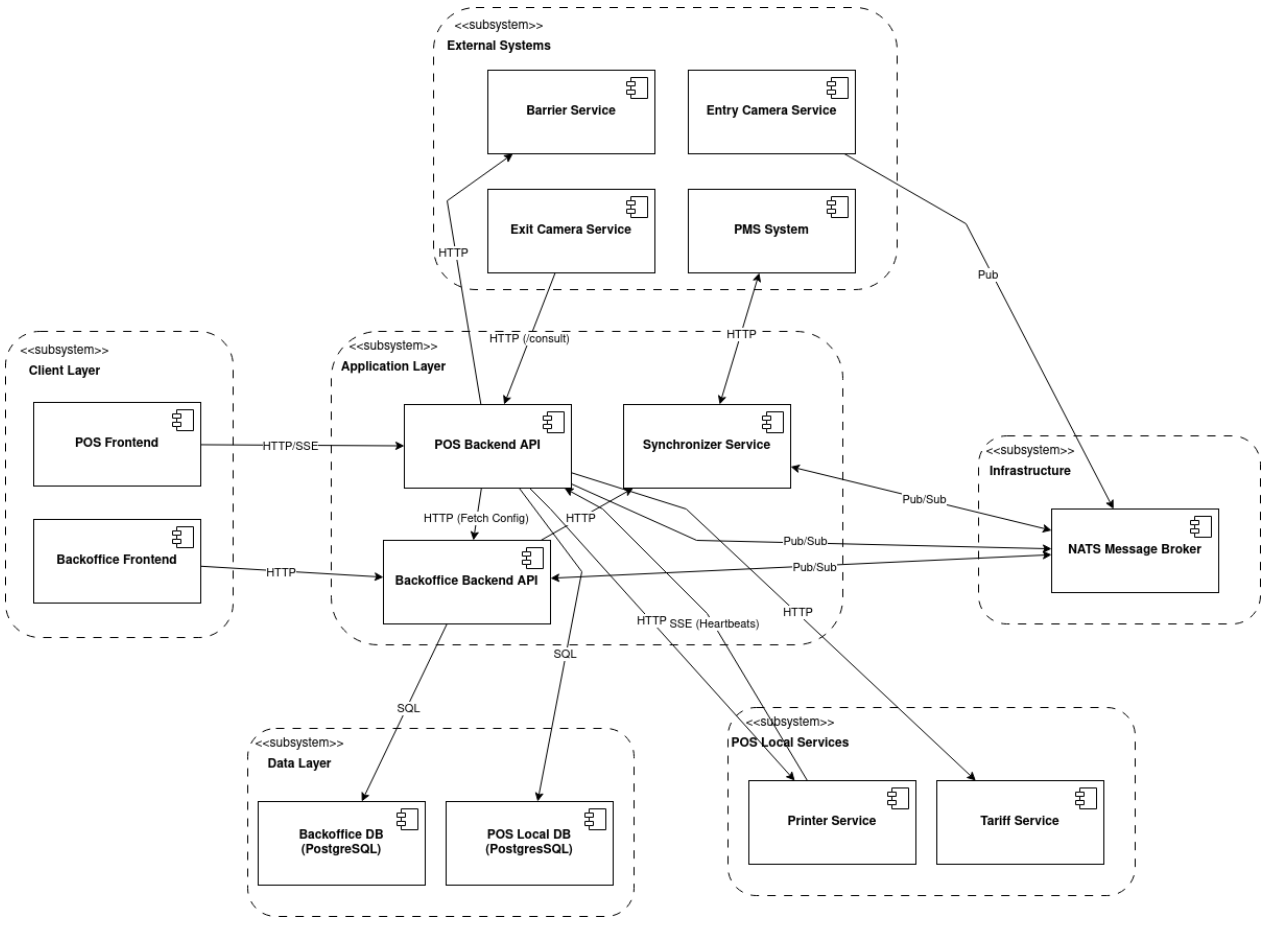

Figure 4.1: Global architecture of the proposed system

4.2.1 POS Architecture

The POS backend follows a three-layer structure: handlers receive incoming HTTP requests and expose SSE endpoints for real-time frontend communication; services contain the core business logic split across authentication, shift management, ticket search, payment, configuration, printing, and barrier control; repositories handle all database interactions with the local PostgreSQL instance.

On the messaging side, the Shift Service publishes `shift.opened` and `shift.closed` events to NATS. The Payment Service publishes `transaction.created` and `ticket.exited` after a successful payment. The Ticket/Search Service subscribes to `ticket.exited` to stay consistent with other terminals. The Config Service subscribes to `pos.config.{terminal_id}` to receive configuration updates from the backoffice.

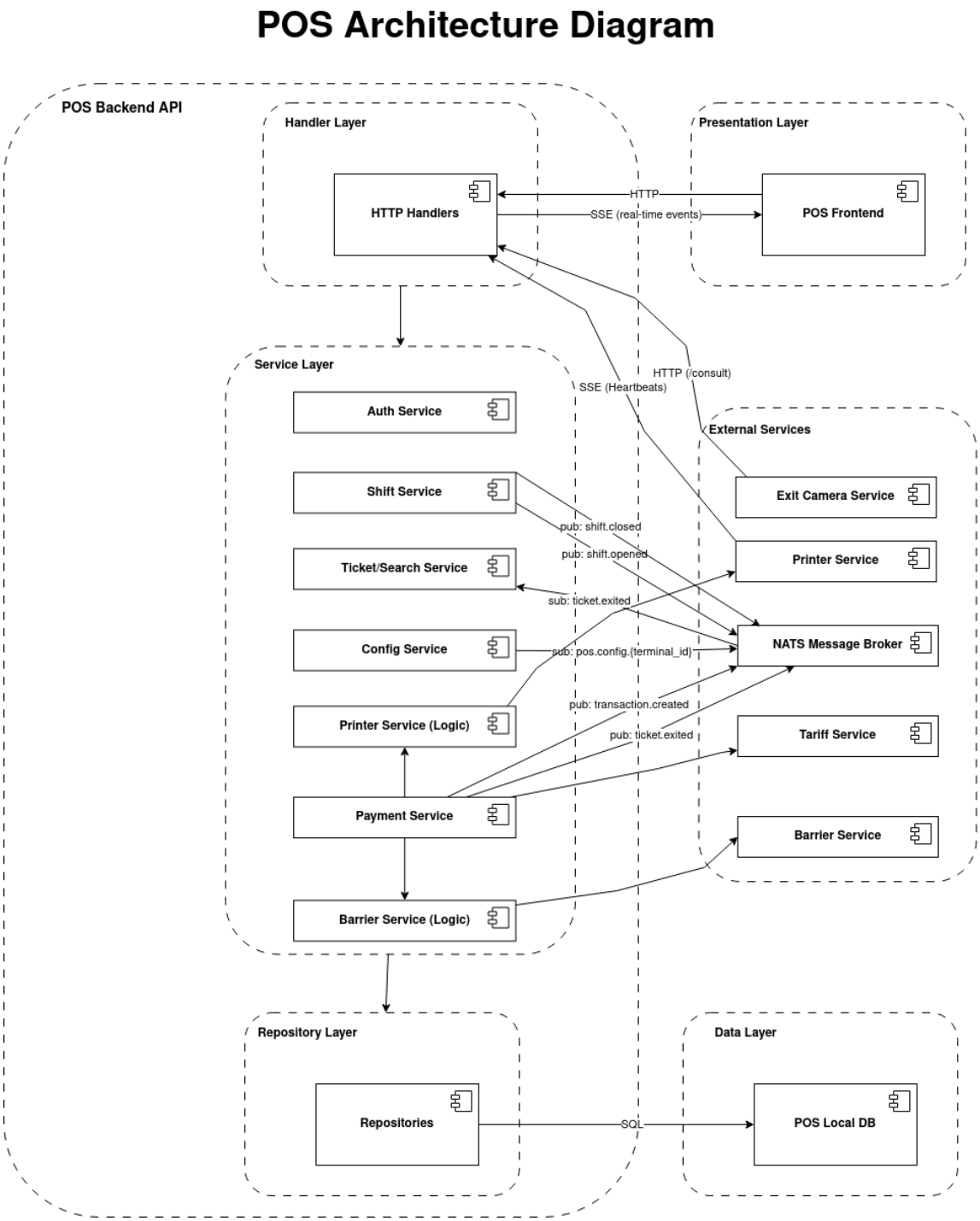

Figure 4.2: POS architecture component diagram

4.2.2 Backoffice Architecture

The backoffice backend uses the same layered structure. The service layer covers all administrative operations along with two dedicated synchronization services: the Shift Sync Service subscribes to `shift.opened` and `shift.closed` and forwards shift data to the PMS; the Transaction Sync

Service subscribes to `transaction.created` and does the same for completed transactions. The POS Terminal Management service publishes `pos.config.{terminal_id}` events whenever a terminal configuration is updated, allowing the concerned terminal to self-update without any direct call.

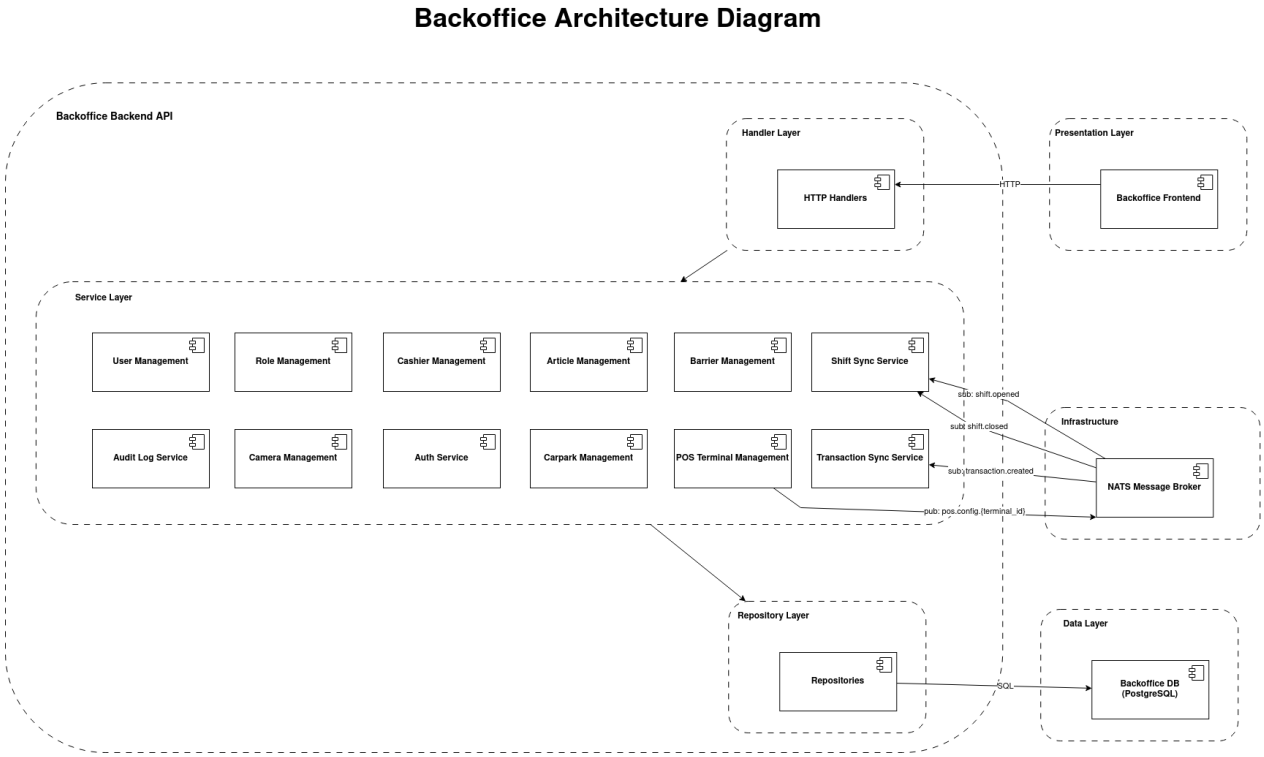

Figure 4.3: Backoffice architecture component diagram

4.3 Performance-Oriented Design Decisions

One of the most important context elements of this project is its intended deployment target. The system is being prepared for the Cairo International Airport, a client of Asteroidea, whose parking facility processes approximately two million transactions per month. At this scale, response time and system reliability are not optional; they are essential requirements. This is the main reason why the previous PHP-based solution was replaced, and why Go was selected as the backend language for this project.

4.3.1 Concurrency with Goroutines

Go's concurrency model is based on goroutines, which are lightweight execution units managed by the Go runtime. Unlike threads, goroutines have a very small memory footprint and can be launched in large numbers without significant overhead. This makes them particularly well suited for a system

that needs to handle multiple simultaneous operations efficiently.

In this project, goroutines are used in several places to avoid blocking the main execution flow. For example, after a payment is confirmed, the receipt printing and the barrier opening are triggered concurrently rather than one after the other. This means the cashier does not wait for one operation to finish before the other starts, which noticeably reduces the total time needed to complete a vehicle exit.

Beyond that, the system runs several background goroutines in parallel at all times: handling incoming camera requests, maintaining NATS subscriptions for real-time data exchange, and broadcasting events to connected frontends. Each of these runs independently without interfering with the others, which keeps the system responsive even under concurrent load.

4.3.2 Real-Time Communication with SSE

Server-Sent Events (SSE) were chosen over polling for two specific communication channels in the system.

The first is between the POS backend and the cashier interface. When a vehicle is detected at the exit, the camera service triggers an endpoint on the backend, which immediately pushes the vehicle and billing information to the frontend through an open SSE connection. This allows the cashier's screen to update instantly without any manual refresh or repeated requests.

The second channel is between the POS backend and the printer service. Instead of periodically asking the printer whether it is available, the backend maintains an SSE connection that receives real-time status updates, including paper level, error states, and availability. This reduces unnecessary network traffic and ensures that the system always reflects the actual printer state without delay.

Together, these decisions contribute to a system that is not only functionally complete but also built to perform reliably at the scale required by its deployment environment.

4.4 Integration with External Services

4.4.1 Parking Management System (PMS)

The POS and backoffice environments exchange operational data with the PMS through asynchronous and synchronous communication paths.

- **Presence tickets:** Entry information is produced by the entry camera and published on NATS. POS terminals subscribe to the corresponding subject in order to receive vehicle entry data in real time.
- **Transactions:** After a successful payment, the POS terminal publishes the transaction and the updated ticket state on NATS. These events are consumed by the backoffice environment, which forwards them to the PMS through the synchronizer service.
- **Shift operations:** Shift opening and closing events are also published by the POS terminal and handled by the backoffice synchronization flow before being transmitted to the PMS.

4.4.2 Camera System

The camera system is responsible for detecting vehicles and providing their identification data.

- The exit camera communicates with the POS backend through HTTP requests.
- When a vehicle is detected, the camera triggers the `/consult` endpoint on the POS system.
- If the camera does not provide a valid result, the cashier can perform a manual search using the POS interface.
- The backoffice system can also query camera endpoints to verify their availability and status.

4.4.3 Printer System

The printer system is used to generate receipts and reports.

- The POS backend sends HTTP requests containing either HTML content or generated images to be printed.

- The printer service exposes endpoints to check its status.
- Real-time status updates can also be received using Server-Sent Events (SSE), allowing the POS to monitor printer availability.

4.4.4 Barrier System

The barrier system controls the physical access of vehicles at the parking exit.

- The POS system communicates with the barrier service using HTTP requests.
- After a successful payment, the POS automatically triggers the barrier opening.
- In case of issues, authorized users can manually control the barrier from the POS or backoffice.

4.4.5 Tariff Service

The tariff service is responsible for calculating parking fees based on vehicle data and parking duration.

- The POS backend sends HTTP requests to the tariff service during the payment process.
- The service returns the calculated amount, which is then presented to the cashier.
- If the tariff service is unavailable, the payment process cannot proceed normally.

4.4.6 Configuration Synchronization

Configuration updates are managed from the backoffice and propagated to the concerned POS terminals.

- The backoffice publishes configuration update events on NATS for the targeted POS terminal.
- The POS terminal subscribes to its dedicated configuration subject.
- After receiving the notification, the POS retrieves the updated configuration from the backoffice through HTTP.

4.4.7 Failure Handling and System Behavior

The system is designed to handle partial failures in external services while maintaining core functionality.

- **Camera failure:** The cashier can manually search for a ticket using vehicle information.
- **Tariff service failure:** The system cannot proceed with payment and switches to an unavailable state.
- **Printer failure:** The payment process continues normally, as printing is not critical for vehicle exit.
- **Barrier issues:** Manual intervention is possible depending on user permissions.

4.5 User Interfaces and Screenshots

4.5.1 POS System Interfaces

The POS system is designed to support the cashier during the vehicle exit and payment process. The following screens illustrate the main steps of this workflow.

4.5.1.1 Initial Setup

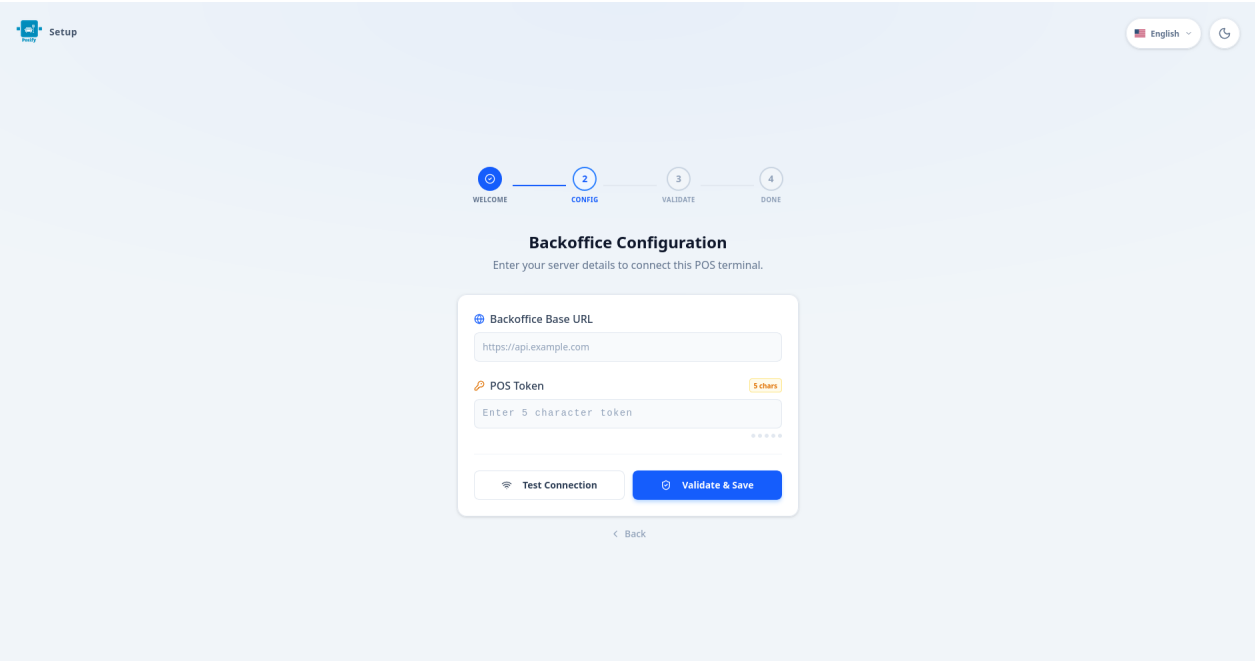

Figure 4.4: POS initial setup page

This interface is displayed when the POS terminal is launched for the first time. It allows the configuration of the connection with the backoffice system by providing the base URL and the terminal token.

4.5.1.2 Authentication

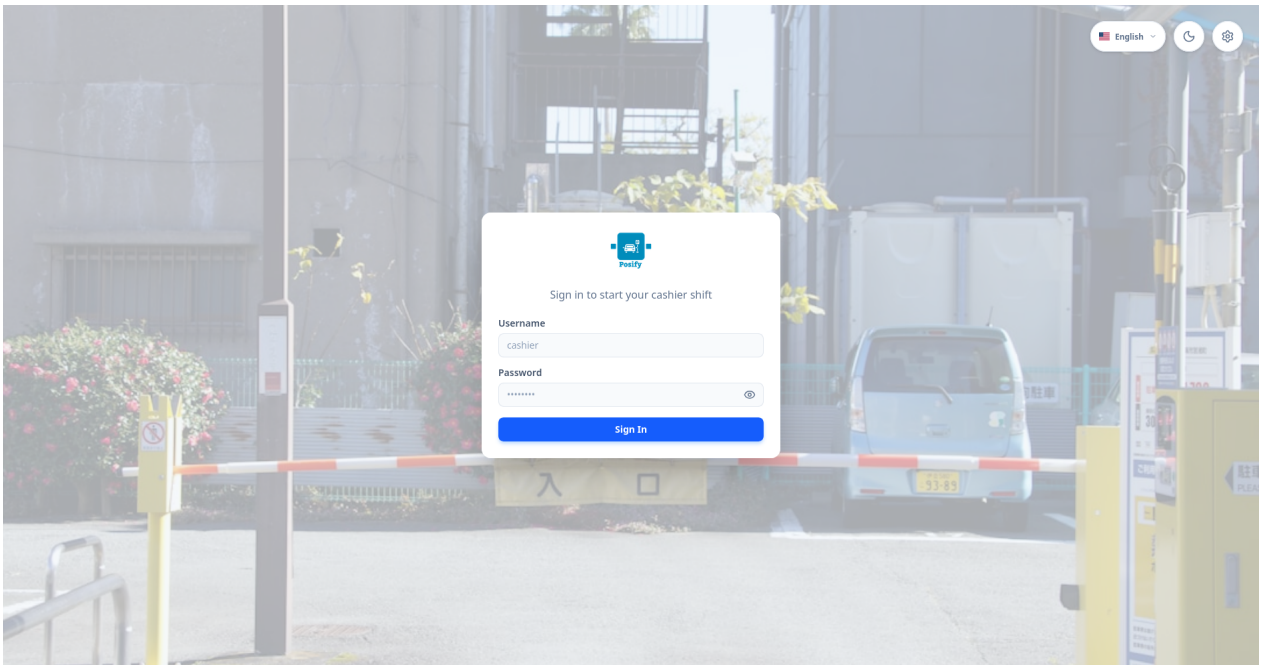

Figure 4.5: POS login page

The login page enables the cashier to authenticate using their credentials before accessing the system.

4.5.1.3 Shift Initialization

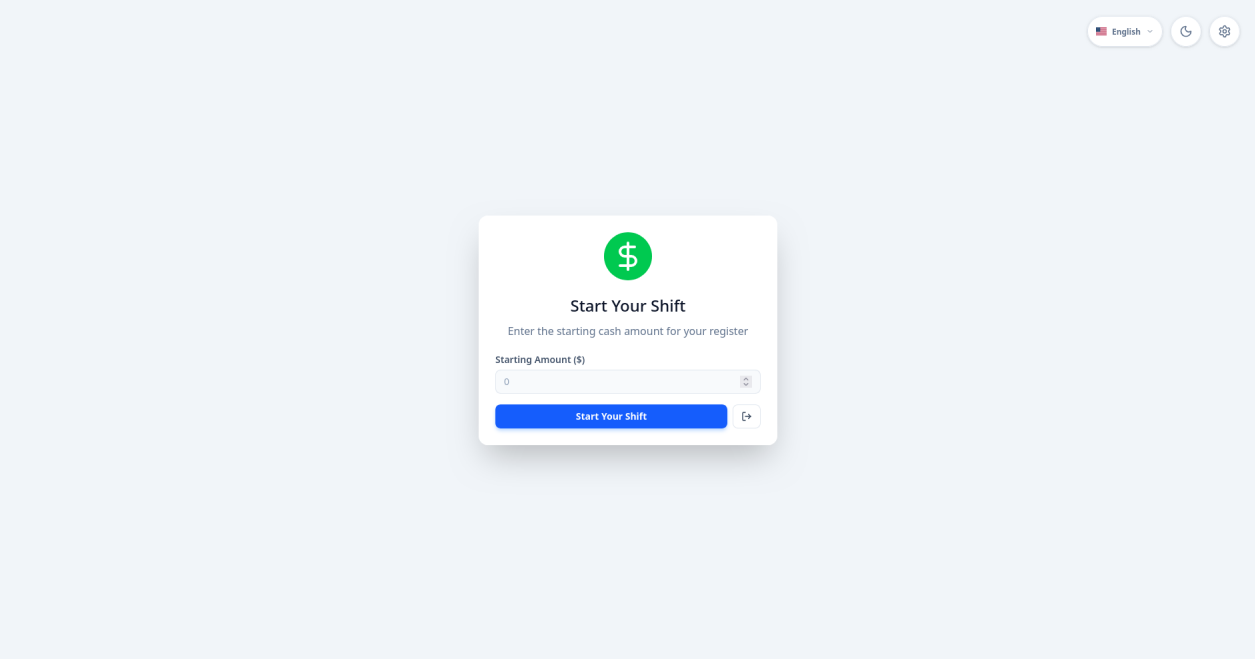

Figure 4.6: Start shift interface

This screen allows the cashier to start a new shift.

4.5.1.4 Main POS Interface

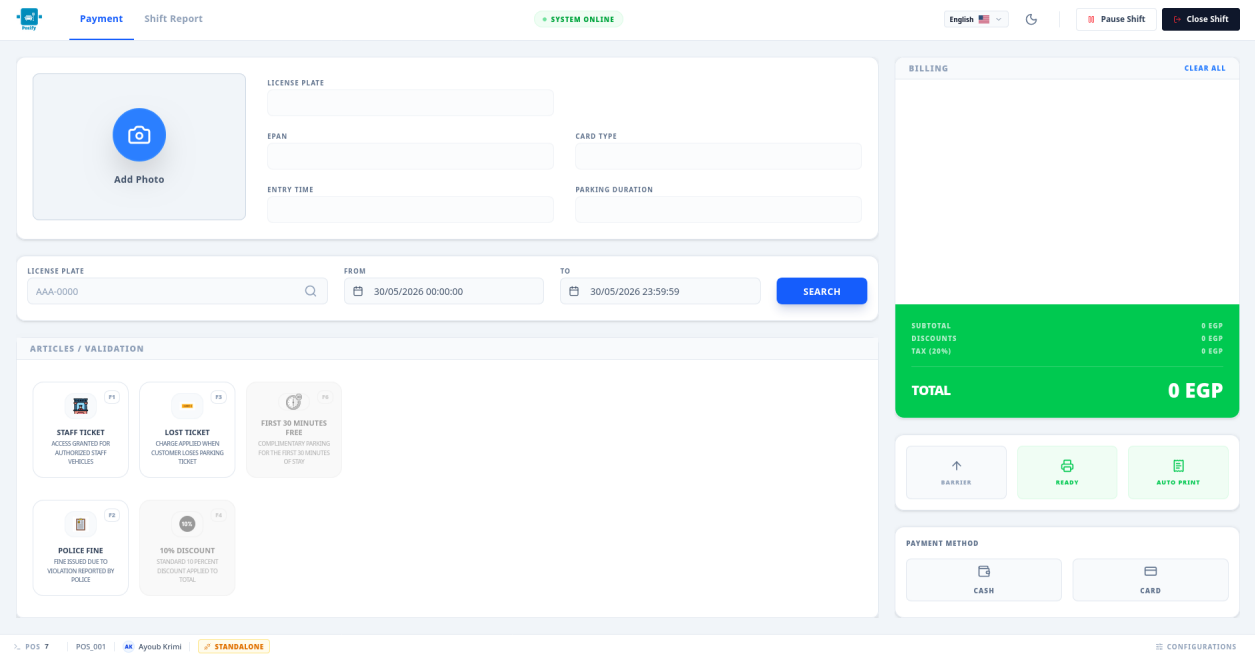

Figure 4.7: Main POS interface

The main interface centralizes all operations related to vehicle processing. It displays vehicle information, billing details, system status indicators, and provides access to features such as manual ticket search and barrier control.

4.5.1.5 Payment Processing State

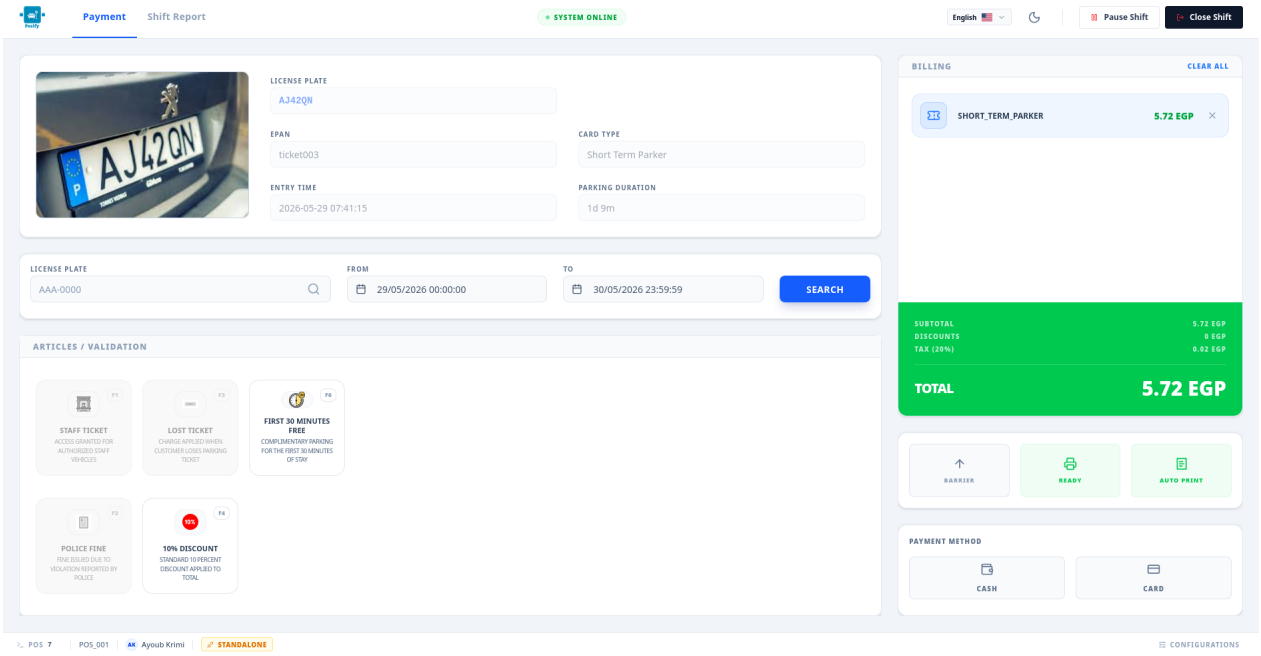

Figure 4.8: Payment processing view

This view represents the state where a vehicle has been detected or manually selected, and the system displays the calculated amount along with the relevant details required to complete the payment.

4.5.2 Backoffice Interfaces

The backoffice platform is used by administrators to configure and manage the system components such as users, roles, POS terminals, and parking configurations.

4.5.2.1 Management Interface

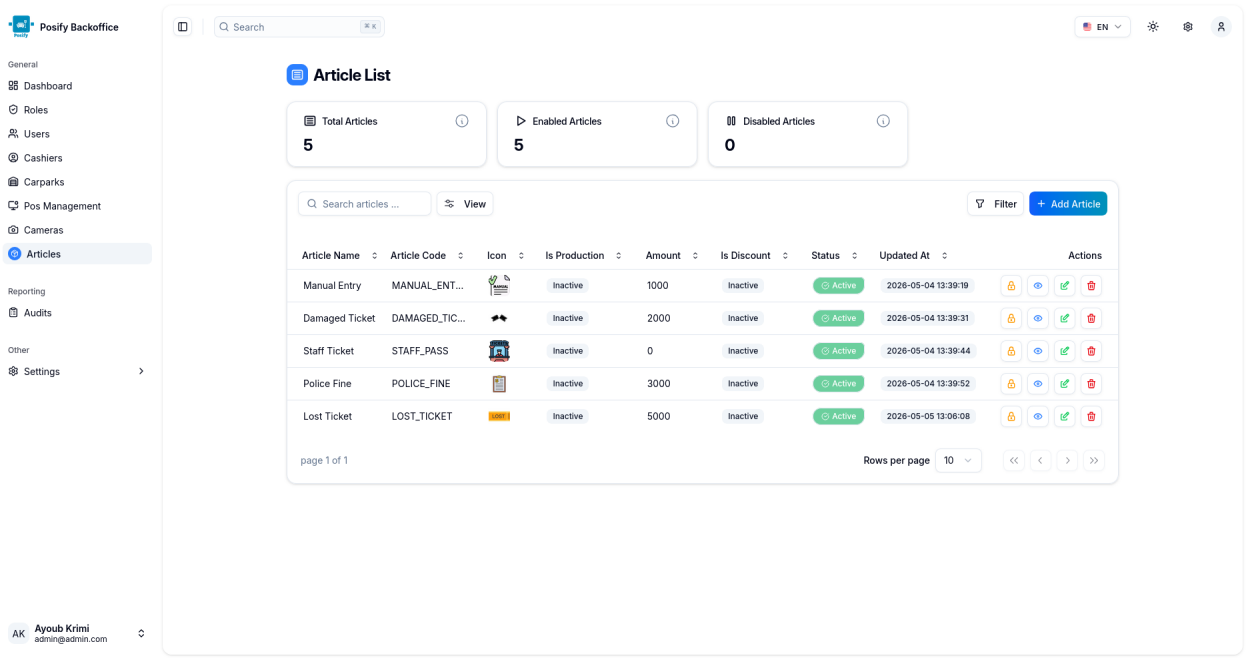

Figure 4.9: Backoffice management page example

This interface illustrates a typical management page, where data is presented in a tabular format. Administrators can view, create, update, or delete entities using the available actions.

4.5.2.2 CRUD Modal Interaction

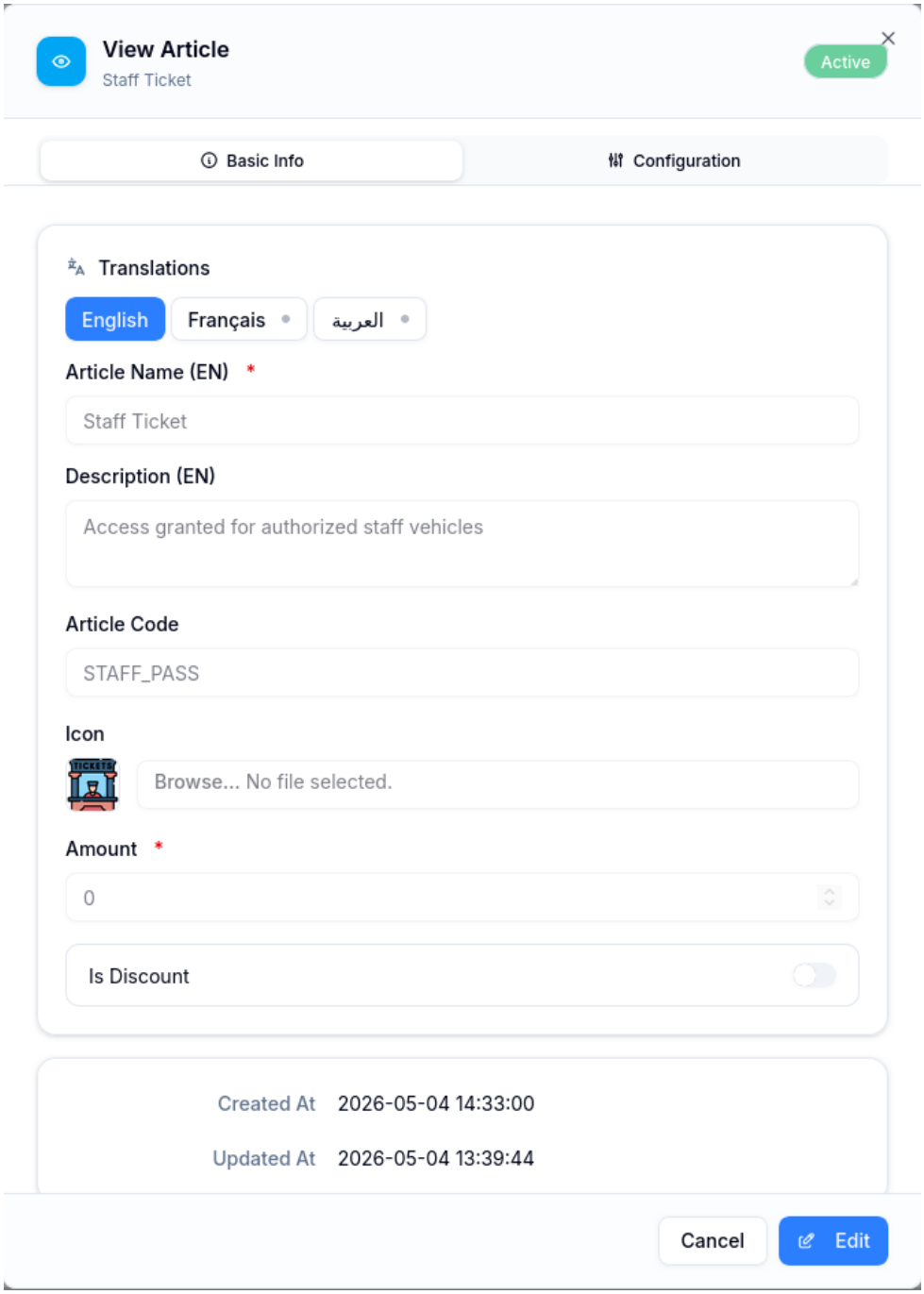

Figure 4.10: Backoffice CRUD modal example

This modal represents the interaction used to create or edit system entities. It demonstrates how the platform handles data input and validation in a structured and user-friendly way.

4.6 Manual Testing and Validation

4.6.1 Testing Approach

The validation process was based on simulating real parking operations. For example, vehicle exit scenarios were reproduced by presenting license plate numbers to the camera system, which triggered the `/consult` endpoint and initiated the payment workflow.

Tests were performed both in a local development setup and within the company environment in order to confirm that the system behaves consistently across different deployment contexts.

4.6.2 POS System Validation

Several key scenarios were tested to validate the POS system:

- **Full payment workflow:** The complete process from vehicle detection to payment validation and barrier opening was successfully executed.
- **Manual ticket search:** When automatic detection failed or returned no result, the cashier was able to manually search for a ticket using the provided interface.
- **Online and standalone modes:** The system was tested in both connected mode (with PMS integration) and standalone mode to ensure continuity of operations.

4.6.3 External Services Testing

The interaction with external services was also validated under different conditions:

- **Camera system:** Scenarios where the camera does not respond or detects an unknown license plate were tested. In such cases, the system correctly redirected the cashier to manual search.
- **Tariff service:** Failures in tariff calculation were simulated to verify that the system prevents payment processing when pricing data is unavailable.
- **Printer system:** Different printer states were tested, including paper out, near-end paper, and offline status. The system handled these cases without blocking the main payment workflow.

4.6.4 Data and Integration Validation

The correctness of data handling and system integration was verified through the following checks:

- Transactions were correctly stored in the local POS database.
- Transaction data was successfully published and synchronized through the messaging system.
- Retrieved vehicle and ticket information was consistent with expected values.

4.6.5 Backoffice Validation

The backoffice platform was tested to ensure proper access control and configuration management:

- Role-based access control was validated by assigning different permissions to users.
- Restricted users were unable to access unauthorized sections.
- Read-only permissions were correctly enforced, allowing data visualization without modification.

Conclusion

This chapter presented the implementation and realization of the developed solution, covering the development environment, the implemented system architecture, and the integration with external services. It also illustrated the main user interfaces of both the POS and backoffice platforms, along with the manual testing and validation activities performed during the internship.

Special attention was given to performance-oriented design choices, particularly the adoption of Go for concurrency handling, the use of event-driven communication with NATS, and real-time interactions through Server-Sent Events (SSE). These decisions were important to ensure that the solution remains reliable and scalable for large parking environments such as Cairo International Airport.

Overall, the implemented system provides a modern, modular, and maintainable parking management solution adapted to both operational and administrative needs.

General Conclusion and Perspectives

This internship project was carried out at Asteroidea with the objective of designing and developing a modern parking management solution composed of a Point of Sale (POS) system and a backoffice platform. The work addressed clear limitations identified in the existing system, particularly the lack of dynamic configuration and the complexity of deployment across different environments.

The main achievements of this project include the implementation of a fully functional POS interface for cashiers, its corresponding backend API, and a backoffice platform with its own backend. The system also includes printing capabilities for receipts and reports, supports real-time communication through Server-Sent Events, asynchronous messaging via NATS, and integrates with several external services including the Parking Management System, barrier control, camera detection, and tariff calculation.

The core features of the solution have been completed and validated through manual testing in both local and company environments. The results confirmed that the system handles normal operations as well as failure scenarios in a reliable and consistent manner. However, the solution has not yet been fully deployed in a production client environment, which represents the natural next step following this internship.

In terms of perspectives, several improvements and extensions can be considered for future development. First, automated testing could be introduced to complement the current manual validation approach and improve code reliability over time. Second, the system could be extended to support additional parking management systems beyond the current integration. Third, more advanced monitoring and reporting features could be added to the backoffice platform to provide administrators with deeper operational insights. Finally, a more refined offline mode with conflict resolution mechanisms could further strengthen system availability in degraded network conditions.

Overall, this project provided a solid foundation for a flexible and maintainable parking solution, and its modular architecture ensures that it can be extended and adapted to meet future operational requirements.

Bibliography

[1] "Go documentation." [Accessed on 22-April-2026]. [Online]. Available: https://go.dev/doc/

[2] "Gin web framework documentation." [Accessed on 22-April-2026]. [Online]. Available: https://gin-gonic.com/en/docs/

[3] "Bun orm documentation." [Accessed on 22-April-2026]. [Online]. Available: https://bun.uptrace.dev/

[4] "Zerolog repository." [Accessed on 22-April-2026]. [Online]. Available: https://github.com/rs/zerolog

[5] "RFC 7519 - json web token." [Accessed on 22-April-2026]. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc7519

[6] "Swaggo repository." [Accessed on 22-April-2026]. [Online]. Available: https://github.com/swaggo/swag

[7] "PostgreSQL documentation." [Accessed on 22-April-2026]. [Online]. Available: https://www.postgresql.org/docs/

[8] "NATS documentation." [Accessed on 22-April-2026]. [Online]. Available: https://docs.nats.io/

[9] "Docker documentation." [Accessed on 22-April-2026]. [Online]. Available: https://docs.docker.com/

[10] "React documentation." [Accessed on 22-April-2026]. [Online]. Available: https://react.dev/

[11] "TypeScript documentation." [Accessed on 22-April-2026]. [Online]. Available: https://www.typescriptlang.org/docs/

[12] "Tailwind CSS documentation." [Accessed on 22-April-2026]. [Online]. Available: https://tailwindcss.com/docs

[13] "shadcn/ui documentation." [Accessed on 22-April-2026]. [Online]. Available: https://ui.shadcn.com/docs

[14] "Zod documentation." [Accessed on 22-April-2026]. [Online]. Available: https://zod.dev/

[15] "Zustand documentation." [Accessed on 22-April-2026]. [Online]. Available: https://zustand.docs.pmnd.rs/

[16] "TanStack Router documentation." [Accessed on 22-April-2026]. [Online]. Available: https://tanstack.com/router/latest/docs/overview

[17] "TanStack Query documentation." [Accessed on 22-April-2026]. [Online]. Available: https://tanstack.com/query/latest

[18] "EndeavourOS documentation." [Accessed on 5-May-2026]. [Online]. Available: https://endeavouros.com/

[19] "i3 window manager documentation." [Accessed on 5-May-2026]. [Online]. Available: https://i3wm.org/docs/

[20] "GitLab CI/CD documentation." [Accessed on 5-May-2026]. [Online]. Available: https://docs.gitlab.com/ci/

[21] "Nexus Repository documentation." [Accessed on 5-May-2026]. [Online]. Available: https://help.sonatype.com/en/sonatype-nexus-repository.html

[22] "Portainer documentation." [Accessed on 5-May-2026]. [Online]. Available: https://docs.portainer.io/

[23] "Neovim documentation." [Accessed on 5-May-2026]. [Online]. Available: https://neovim.io/doc/user/

[24] "Postman documentation." [Accessed on 22-April-2026]. [Online]. Available: https://learning.postman.com/docs/introduction/overview/

[25] "cURL documentation." [Accessed on 5-May-2026]. [Online]. Available: https://curl.se/docs/

[26] "DBeaver documentation." [Accessed on 5-May-2026]. [Online]. Available: https://dbeaver.com/docs/

Appendices

Appendix A. Carbon Footprint Estimation (Bilan Carbone)

This appendix presents an estimation of the carbon footprint generated by the activities carried out during this internship project, following the guide provided by Institut Supérieur d'Informatique (ISI), Université Tunis El Manar. The objective is to identify, quantify, and reflect on the greenhouse gas emissions (expressed in kg CO_2 equivalent) associated with the main activities of the project.

I. Scope of the Analysis

- **Period:** February 2026 to May 2026 (4 months)
- **Location:** Local internship in Tunisia (no international travel)
- **Emission categories covered:**
  1. Transport and commuting
  2. IT hardware (manufacturing footprint)
  3. Infrastructure and code (energy usage, cloud, AI tools)
  4. Office materials (printing)
- **Unit:** Kilograms of CO_2 equivalent (kg CO_2e)

II. Estimation by Category

1. Transport and Commuting

The internship lasted approximately 18 weeks at 4 days per week, totaling 72 commuting days. The daily route consists of walking, taxi, and train, covering approximately 16.3 km one way (32.6 km round trip).

- Taxi segment: 8.3 km each way $\times$ 2 $\times$ 72 days = 1,195.2 km

  Emissions = $1195.2 \times 0.20 = \mathbf{239.0}$ kg CO_2e

- Train segment: 8 km each way × 2 × 72 days = 1,152 km

  Emissions = $1152 \times 0.004 = \mathbf{4.6}$ kg CO_2e

- Walking: 0 kg CO_2e

Transport subtotal: 243.6 kg CO_2e

2. IT Hardware

The laptop used is a personal existing machine (MSI GF63 Thin 11SC, 15-inch gaming laptop). Its estimated manufacturing footprint is approximately 300 kg CO_2e. The fraction attributed to this project is calculated over an assumed device lifespan of 4 years (48 months).

$$\text{Emissions} = 300 \times \frac{4}{48} = \mathbf{25.0} \text{ kg } CO_2e$$

3. Infrastructure and Code

A. Local PC energy consumption

The laptop was used approximately 8 hours per day over 4 months (≈ 704 hours total). At an average power of 0.065 kW and using the Tunisian electricity emission factor of 0.545 kg CO_2e/kWh:

$$\text{Emissions} = 0.065 \times 704 \times 0.545 = \mathbf{24.9} \text{ kg } CO_2e$$

B. Company local server

The project used the company's internal server for CI/CD pipeline execution and backoffice deployment testing. Estimated at 0.1 kW over approximately 300 hours:

$$\text{Emissions} = 0.1 \times 300 \times 0.545 = \mathbf{16.4} \text{ kg } CO_2e$$

C. AI tools usage

AI assistants (Claude and ChatGPT) were used regularly throughout the project, averaging approximately 7 to 8 prompts per day over 72 working days (≈ 540 queries). Using an average emission factor of 6 g CO_2e per query (medium-length responses):

$$\text{Emissions} = 540 \times 0.006 = \mathbf{3.2} \text{ kg } CO_2e$$

D. Data transfers

Development activities involved significant data transfers including npm and Go module downloads, Docker image builds and pushes through the CI/CD pipeline, Git operations, and API testing. Total estimated volume: ≈ 95 GB.

$$\text{Emissions} = 95 \times 0.02 = \mathbf{1.9} \text{ kg } CO_2e$$

Infrastructure subtotal: 46.4 kg CO_2e

4. Office Materials (Printing)

The final report contains 88 pages in total, including 2 cover pages printed separately on a single cover sheet. The remaining 86 pages are printed single-sided across 5 copies, totaling 430 sheets (≈ 2.15 kg of paper).

- Paper emissions: $2.15 \times 1.3 = \mathbf{2.8}$ kg CO_2e
- Toner (estimated fraction): ≈ **0.6** kg CO_2e

Printing subtotal: 3.4 kg CO_2e

III. Summary

Tableau 5.1: Carbon footprint summary

Emission Category	**Calculation Detail**	**Estimated (kg CO_2e)**	**%**
1. Transport	Taxi (1,195.2 km) + Train (1,152 km)	243.6	76.5%
2. IT Hardware	MSI GF63, 4/48 months fraction	25.0	7.9%
3. Infrastructure & Code	PC usage + local server + AI tools + data transfers	46.4	14.6%
4. Office Materials	430 sheets, 5 copies, single-sided	3.4	1.1%
TOTAL		**318.4**	**100%**

IV. Reduction Proposals

Based on this analysis, the dominant emission source is daily commuting, accounting for over three quarters of the total footprint. The following actions could meaningfully reduce the carbon impact of a similar future project.

- **Transport:** The taxi segment alone represents the largest single contributor. Replacing it with public transport or cycling where feasible would have the most significant impact. Even partial remote work days would noticeably reduce this category.
- **Digital sobriety:** Limiting unnecessary Docker image rebuilds, cleaning up unused dependencies, and batching commits rather than pushing per small change would reduce both data transfer and server load.
- **AI usage:** Being more selective with AI queries and preferring concise prompts reduces the per-session footprint, which accumulates over months of daily use.
- **Printing:** Limiting the number of printed copies and using recycled paper would reduce the office materials footprint, which is already minor but avoidable.

V. Reflection

The most critical emission source in this project is transport, and more specifically the taxi segment, which alone accounts for the majority of the total footprint. This is tied directly to the physical commute structure rather than any technical decision, which makes it a useful reminder that the carbon impact of a software project is not limited to code and infrastructure.

From a development standpoint, the infrastructure category is still notable. The combination of local server usage, AI tool consumption, and data-intensive workflows accounts for nearly 15% of the total, a figure that is easy to overlook when the work feels purely digital.

One relevant design choice in this project is the use of Go as the backend language. Go is known for its efficiency in terms of CPU usage and memory consumption compared to interpreted languages. At scale, particularly given that this system targets a facility processing two million transactions per month, a more energy-efficient backend directly translates into reduced server energy consumption over the lifetime of the product. As a result, the carbon impact of this project extends well beyond the internship period itself.

This exercise highlights that software development, even when entirely local and infrastructure-light, carries a real environmental cost. Building awareness of this from the first professional project is a step toward more responsible engineering practice.

ملخّص

يُندرج هذا المشروع في إطار تربص تم إنجازه داخل شركة Asteroidea. يتمثل الهدف الرئيسي في تطوير نظام متكامل لإدارة عمليات مواقف السيارات، يجمع بين منصة Point Of Sale (POS) ومنصة Backoffice للإدارة والإشراف. يعتمد النظام على معالجة آنية وتواصل في الزمن الحقيقي، مع تكامل مع تجهيزات مواقف السيارات مثل الكاميرات والحواجز والطابعات. كما يوفر معالجة عمليات الدفع، مزامنة المعطيات، وإدارة الصلاحيات.

تم تطوير المشروع باستخدام تقنيات حديثة مثل Go وReact وPostgreSQL وNATS وTypeScript.

كلمات مفاتيح : نقاط البيع، إدارة مواقف السيارات، Golang، الأنظمة الآنية، Backoffice

Résumé

Ce projet, intitulé *ParkFlow POS*, a été réalisé dans le cadre d'un stage au sein de l'entreprise *Asteroidea*. Il consiste à développer un système intégré de gestion des opérations de parking combinant une plateforme *Point Of Sale (POS)* et une plateforme *Backoffice*.

Le système supporte la communication temps réel et les échanges orientés événements, tout en intégrant plusieurs équipements de parking tels que les caméras, les barrières et les imprimantes. Il permet également la gestion des paiements, la synchronisation des données et le contrôle des accès.

Le projet a été développé avec les technologies *Go*, *React*, *PostgreSQL*, *NATS* et *TypeScript*.

Keywords : Point Of Sale, Gestion de parking, Golang, Systèmes temps réel, Backoffice

Abstract

This project, entitled *ParkFlow POS*, was developed during an internship at *Asteroidea*. Its main objective is to design and implement an integrated parking management solution combining a *Point Of Sale (POS)* platform and a *Backoffice* platform.

The system supports real-time communication and event-driven interactions while integrating parking-related hardware such as cameras, barriers, and printers. It also provides payment handling, data synchronization, and access control features.

The project was developed using technologies including *Go*, *React*, *PostgreSQL*, *NATS*, and *TypeScript*.

Keywords : Point Of Sale, Parking Management, Golang, Real-Time Systems, Backoffice

2 نهج أبو الريحان الباروني 2080 أريانة الهاتف : 71 706 164 الفاكس : 71 706 698 البريد الالكتروني : isi@isim.rnu.tn

2, Abou Raihane Bayrouni 2080 l'Ariana **Tél** : 71 706 164 **Fax** : 71 706 698 **Email** : isi@isim.rnu.tn